

Documentação do Programador - Orbbec

Projeto Bolsas Trust 26 - Julia Koene Moreira da Silva

Index

1. Instalação
2. Resumo de funcionamento
3. Arquivos
4. runner.py
5. utils.py
6. locale_setup.py
7. config.py
8. ui/draw.py
9. ui/exercise_intro.py
10. ui/forms.py
11. ui/menu.py
12. ui/theme.py
13. ui/view_data.py
14. exercises/sit_and_reach.py
15. exercises/back_scratch.py
16. locale/
17. camera.py

Instalação

1. Faça o download e instale:
 - a. **Orbbec SDK v2 para Windows** ([OrbbecSDK_v2.8.7_win64.exe](#)).
 - b. **Anaconda** ([Anaconda Download](#)).
2. Teste o funcionamento pelo **OrbbecViewer** (instalado com o SDK). Utilize uma porta **USB 3.0** com o cabo USB-C original da câmara (cabos USB-C para USB-A não originais podem falhar). A Femto Mega pode necessitar de alimentação externa.
3. Faça a cópia do **GitHub** (branch [feature/orbbec-femto-mega](#) [GitHub do Projeto](#)). Tenha atenção com o tamanho do caminho da pasta, se for muito longo pode dar erro quando for rodar.
4. Configure o ambiente virtual **Anaconda**, no caminho do arquivo no programa da Anaconda e execute os comandos seguintes:

```
conda create -n propicie_env python=3.8
conda activate propicie_env
```

5. Instale as bibliotecas no ambiente virtual:

```
pip install numpy==1.23.5
```

```
pip install mediapipe==0.10.9
pip install opencv-python pandas openpyxl
pip install --upgrade pyorbbeccsdk2
```

6. Compile as traduções para que não hajam erros, também sempre que forem alteradas, no ambiente virtual:

```
pybabel compile -i locale/en_US/LC_MESSAGES/messages.po -o
locale/en_US/LC_MESSAGES/messages.mo
pybabel compile -i locale/pt_PT/LC_MESSAGES/messages.po -o
locale/pt_PT/LC_MESSAGES/messages.mo
```

Resumo de funcionamento

O sistema visa validar o uso e método dos sensores RGB do Orbbec Femto Mega e MediaPipe Holistic para deteção de landmarks no corpo humano para avaliação de flexibilidade de idosos.

Arquitetura Geral

Quando rodar o programa a primeira tela será de menu com as opções:

- **Automático:** Sentar e Alcançar > Alcançar atrás das Costas.
- **Sentar e Alcançar:** Somente este exercício.
- **Alcançar atrás das Costas:** Somente este exercício.
- **Visualizar Dados:** Visualização gráfica dos dados salvos em tabelas excel.

Cada exercício, independente se for no automático, passará sempre por:

- **Registo do participante:** formulário (idade, altura, peso, género).
- **Ecrã de introdução:** mostra o progresso das repetições com círculos, com X verde - concluído, com bola amarela - próximo exercício e inteira azul - não realizado.
- **Fase de calibração (somente no Sentar e Alcançar):** o utilizador mantém a posição inicial durante 5 segundos.
- **Fase de medição:** o utilizador realiza o exercício mantendo sua posição por alguns segundos até o sistema calcular a distância.
- **Ecrã de distância real:** o operador insere a distância medida manualmente.
- **Ecrã de resultado da repetição:** mostra distância do sistema, distância real, e erro.
- **Ecrã final (após todas as repetições):** tabela com as 4 repetições (2 por lado) e respetivos erros.
- **Retorno ao Menu:** ocorre no fim das repetições ou se o programa for interrompido em qualquer momento.

Pipeline de Deteção

Em cada frame da câmara o sistema:

- Faz a conversão de BGRA para BGR (BlueGreenRed)
- Depois a conversão para RGB (RedGreenBlue)
- Processa a imagem transformada no MediaPipe Holistic.process(rgb)
- Faz a extração das posições de referência (pose, mãos)

- Para Sentar e Alcançar:
 - Calcula os ângulos (joelho, quadril, cotovelo)
 - Verifica a posição na calibração e na postura durante o exercício
- Realiza o cálculo da distância (px → cm)
- Faz a renderização do canvas para mostrar a tela e imagens (header + feed + overlays)

A conversão de pixels para centímetros é feito através de constantes testadas empiricamente definidas no `config.py`.

WindowManager

Todas as janelas do programa são geridas pela classe `WindowManager` em `utils.py`:

- `cv2.namedWindow`: Cria janelas nomeadas;
- `show(canvas)`: Apresenta frames da imagem capturada do Kinect com suas edições;
- `poll()`: Lê `cv2.waitKey` para retornar a tecla pressionada ou se o programa foi fechado;
- `close()`: Fecha a janela.

Tradução

O arquivo `locale_setup.py` exporta `translate()`, as línguas suportadas são português de Portugal e inglês Americano, os arquivos com as traduções linha por linha estão no `locale/` nos arquivos `.po`, `msgid` são como os textos estão no código, alguns em forma de variável, os `msgstr` é a tradução e como vão aparecer nas telas.

Após uma edição nos arquivos de texto é necessário compilá-los como mostrado nos passos de instalação.

Dados dos Exercícios

Os resultados de cada repetição são salvos em um arquivo excel através de `append_to_excel()`, que lê o arquivo com `pandas` e anexa as novas linhas. Também são salvos em um Log CSV através do `append_to_log()` com timestamp.

Para visualização dos dados tem a tela de Visualização de Dados que pode ser aberta pelo menu, é possível selecionar entre as opções e calcular um gráfico com os parâmetros escolhidos, utiliza o `Matplotlib` para geração dos gráficos.

Arquivos

A organização dos arquivos do sistema é atualmente:

```

.
├── /arquivos/
│   ├── relatorios           # Relatórios de progresso e finais detalhados.
│   ├── tabelas_testes      # Planilhas de dados de teste.
│   └── tabelas_utentes     # Planilhas com dados coletados dos testes com usuários.
└── /exercises/
```

— back-scratch.py	# Script Python para o teste de Alcançar atrás das Costas.
— sit-and-reach.py	# Scripts Python para o teste de Sentar e Alcançar.
— /locale/	# Contém os arquivos de texto com suporte de linguagem.
— /ui/	
— draw.py	# Script para criação de telas padronizadas.
— exercise_intro.py	# Script para mostrar qual o próximo exercício e quais foram feitos.
— forms.py	# Cria as janelas/telas de registro, distância real, resultado de repetição e final de exercício.
— menu.py	# Cria o menu e o funcionamento dos botões: Automático, Sentar e Alcançar, Alcançar atrás das Costas, Visualizar Dados e Terminar Sessão.
— theme.py	# Registra as cores e padrões.
— view_data.py	# Cria a tela de visualização de dados com o Matplotlib.
— .gitignore	# Especifica arquivos a serem ignorados pelo Git.
— camera.py	# Captura a imagem da câmera Orbbec.
— config.py	# Dados fixos para os cálculos do sistema.
— locale_setup.py	# Aplica a linguagem utilizando os arquivos em locale.
— runner.py	# Um script Python simples para executar todos os testes.
— utils.py	# Script com todas as funções comuns, que antes eram duplicadas.
— README.md	# Arquivo para mais informações sobre o sistema do projeto.

Runner.py

Esse arquivo é o que executa todo o sistema.

Ao executar fará as importações necessárias e inicializará o Kinect e o Medipipe Holistic;

```
import cv2
from pykinect2 import PyKinectRuntime, PyKinectV2
import mediapipe as mp

from ui.menu import show_main_menu
from utils import ReturnToMenu
from exercises import sit_and_reach, back_scratch

kinect =
PyKinectRuntime.PyKinectRuntime(PyKinectV2.FrameSourceTypes_Color)
holistic = mp.solutions.holistic.Holistic()
```

Função para fechar o programa;

```
def finish():
    cv2.destroyAllWindows()
    raise ReturnToMenu()
```

Entra no loop principal, então ele aguarda uma das opções serem escolhidas, o “quit” está fora do try except pois qualquer tela aberta pelas outras opções devem retornar ao menu quando fechadas, e somente pelo menu é possível sair da aplicação corretamente;

```

while True:
    escolha = show_main_menu()

    if escolha == "quit":
        break

    try:
        if escolha == "auto":
            print("A iniciar Modo Automático...")
            sit_and_reach.run(kinect, holistic, finish)
            print("A iniciar o segundo exercício...")
            back_scratch.run(kinect, holistic, finish)

        elif escolha == "sit_and_reach":
            print("A iniciar Sit and Reach...")
            sit_and_reach.run(kinect, holistic, finish)

        elif escolha == "back_scratch":
            print("A iniciar Back Scratch...")
            back_scratch.run(kinect, holistic, finish)

        elif escolha == "view_data":
            try:
                from ui.view_data import show_data_visualization
                show_data_visualization()
            except Exception as e:
                print(f"[RUNNER] Erro no view_data: {e}", flush=True)
    except ReturnToMenu:
        continue

```

Quando uma opção é selecionada a função dela será chamada, passando de parâmetro, o Kinect e o MediPipe Holistic iniciados, e deixa um print como registro no cmd. Em `elif escolha == "view_data"` há um try catch pois se o programa não encontrar a planilha, ou houver um erro no Matplotlib ele seja reconhecido para ser tratado.

Por fim ele destroi todas as janelas e fecha o Kinect quando o quit é escolhido;

```

cv2.destroyAllWindows()
kinect.close()

```

Utils.py

O arquivo `utils.py` serve para padronizar facilmente as telas e outras funções comuns à ambos os exercícios.

Faz as importações;

```

import math
import datetime as dt

```

```
import cv2
import numpy as np
import pandas as pd
```

Caso haja algum erro ele retorna ao menu;

```
class ReturnToMenu(Exception):
    """Raised when user presses ESC to return to main menu."""
    pass
```

Uma função para fazer com que todas as janelas criadas tenham “CAPACITA -” no nome, para padronizar;

```
def win_title(name):
    """Return window title with CAPACITA prefix."""
    return f"CAPACITA - {name}"
```

Função para fazer o cálculo euclidiano de dois pontos, utilizado em ambos os exercícios, pé - mão, mão - mão;

```
def calculate_distance_2d(point1, point2):
    """Euclidean distance between two (x, y) points."""
    return np.sqrt((point1[0] - point2[0])**2 + (point1[1] - point2[1])**2)
```

Função faz o cálculo dos ângulos do joelho, quadril e cotovelo utilizado em sit_and_reach.py;

```
def calculate_angle(a, b, c):
    """
    Angle (degrees) at vertex *b* formed by segments b→a and b→c.
    Each argument is a 2-element sequence (x, y).
    """
    v1 = (a[0] - b[0], a[1] - b[1])
    v2 = (c[0] - b[0], c[1] - b[1])

    dot = v1[0] * v2[0] + v1[1] * v2[1]
    norm1 = math.sqrt(v1[0]**2 + v1[1]**2)
    norm2 = math.sqrt(v2[0]**2 + v2[1]**2)

    cos_a = max(-1.0, min(1.0, dot / (norm1 * norm2)))
    return math.degrees(math.acos(cos_a))
```

Função faz média aritmética de uma lista, serve para fazer a média da distância em uma janela de 6 frames;

```
def rolling_average(values):
    """Mean of all elements in *values*."""
    return sum(values) / len(values)
```

Função para desenhar os arcos, cria duas linhas e preenche o espaço, serve para criar o visual no exercício de sit_and_reach.py;

```
def draw_angle_arc(image, p1, p2, p3, angle):
```

```

"""
Draw two lines (p1→p2, p2→p3) and a filled arc at vertex p2
that spans *angle* degrees.
"""

cv2.line(image, p1, p2, (255, 255, 0), 2)
cv2.line(image, p2, p3, (255, 255, 0), 2)

p1_arr = np.array(p1)
p2_arr = np.array(p2)
radius = int(np.linalg.norm(p1_arr - p2_arr) / 2)

cv2.ellipse(image, tuple(p2_arr.astype(int)), (radius, radius), 0,
0, angle, (0, 255, 0), -1)
cv2.ellipse(image, tuple(p2_arr.astype(int)), (radius, radius), 0,
0, angle, (0, 0, 255), 2)

```

Função obtém o último frame do Kinect (1920×1080 BGRA), converte para BGR, executa MediaPipe Holistic, utilizado em ambos os exercícios;

```

def read_kinect_frame(kinect, holistic):
    """
    GRAB THE LATEST KINECT COLOUR FRAME, RUN MEDIAPIPE HOLISTIC,
    AND RETURN (BGR_IMAGE, HOLISTIC_RESULTS, RAW_FRAME).
    """

    raw = kinect.get_last_color_frame()
    raw = raw.reshape((1080, 1920, 4)) # BGRA
    bgr = cv2.cvtColor(raw, cv2.COLOR_BGRA2BGR)
    rgb = cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB)
    rgb.flags.writeable = False
    results = holistic.process(rgb)
    rgb.flags.writeable = True
    return cv2.cvtColor(rgb, cv2.COLOR_RGB2BGR), results, raw

```

As funções servem para salvar os dados `append_to_excel` para salvar na planilha excel e `append_to_log` para salvar em CSV;

```

def append_to_excel(filepath, row_dict):
    """Append *row_dict* as a new row to an existing Excel workbook.
    Creates the file with headers if it does not exist."""
    try:
        df = pd.read_excel(filepath, engine="openpyxl")
    except FileNotFoundError:
        df = pd.DataFrame([row_dict])
        df.to_excel(filepath, index=False, engine="openpyxl")
    return df
df = pd.concat([df, pd.DataFrame([row_dict])], ignore_index=True)
df.to_excel(filepath, index=False, engine="openpyxl")

def append_to_log(filepath, *fields):
    """Append a comma-separated log line with a leading timestamp."""
    line = ", ".join(str(f) for f in fields)
    with open(filepath, "a") as fh:
        fh.write(f"{dt.datetime.now()}, {line}\n")

```

Classe para gerenciamentos das janelas/telas, redimensiona, cria o canvas, lê inputs, fecha a janela;

```

class WindowManager:
    """Handles OpenCV window lifecycle: create, show, poll key, close on
    ESC/X."""

    def __init__(self, title, finish_cb=None, delay=1, size=None,
on_mouse=None):
        self.winname = title if title.startswith("CAPACITA") else
win_title(title)
        self.finish_cb = finish_cb
        self.delay = delay
        self._closed = False

        cv2.namedWindow(self.winname, cv2.WINDOW_NORMAL)
        if size:
            cv2.resizeWindow(self.winname, size[0], size[1])
        if on_mouse:
            cv2.setMouseCallback(self.winname, on_mouse)

    @property
    def should_close(self):
        return self._closed

    def show(self, canvas):
        cv2.imshow(self.winname, canvas)

```



```

def poll(self):
    key = cv2.waitKey(self.delay) & 0xFF
    if key == 27 or cv2.getWindowProperty(self.winname,
cv2.WND_PROP_VISIBLE) < 1:
        self.close()
        return "close"
    return key

def close(self):
    if not self._closed:
        self._closed = True
        try:
            cv2.destroyWindow(self.winname)
        except:
            pass

```

Função que transforma imagem em ícone e a dimensiona corretamente;

```

def set_app_icon(window_title=None):
    """Set the CAPACITA logo as the icon for a specific or all OpenCV
    windows."""
    import ctypes, os
    _base = os.path.dirname(os.path.abspath(__file__))
    png_path = os.path.join(_base, "arquivos", "icone-Photoroom.png")
    ico_path = os.path.join(_base, "arquivos", "icone-Photoroom.ico")
    if not os.path.exists(ico_path):
        from PIL import Image as PilImage
        PilImage.open(png_path).save(ico_path, format="ICO", sizes=[(32,
32), (48, 48), (64, 64)])
        hicon = ctypes.windll.user32.LoadImageW(0, ico_path, 1, 0, 0,
0x00000010)
        if hicon:
            if window_title:
                hwnd = ctypes.windll.user32.FindWindowW(None, window_title)
            else:
                hwnd = ctypes.windll.user32.FindWindowW("Main HighGUI
class", None)
            if hwnd:
                ctypes.windll.user32.SetClassLongW(hwnd, -14, hicon)
                ctypes.windll.user32.SendMessageW(hwnd, 0x0080, 0, hicon)
                ctypes.windll.user32.SendMessageW(hwnd, 0x0080, 1, hicon)

```

locale_setup.py

Serve para trocar a linguagem do sistema, sendo a padrão português. Faz os import e pega os paths e configurações dos idiomas;

```
import gettext
import os

_LOCALE_DIR = os.path.join(os.path.dirname(__file__), "locale")
_DOMAIN      = "messages"

SUPPORTED_LANGUAGES = ("pt_PT", "en_US")
_DEFAULT_LANGUAGE   = "pt_PT"
```

Objeto que receberá os textos em msgid e mapeará o msgstr correspondente;

```
_translator = gettext.NullTranslations()
```

Função valida que a lang selecionada existe, carrega as traduções se encontrar o arquivo .mo e guarda os resultados da variável global `_translator()`;

```
def set_language(lang: str) -> None:
    """
    Load translations for *lang*.
    Falls back to NullTranslations (returns msgid unchanged) if not
    found.
    """
    global _translator

    if lang not in SUPPORTED_LANGUAGES:
        raise ValueError(f"Unsupported language '{lang}'. Choose from
        {SUPPORTED_LANGUAGES}")

    try:
        _translator = gettext.translation(
            domain      = _DOMAIN,
            localedir= _LOCALE_DIR,
            languages= [lang],
        )
    except FileNotFoundError:
        # .mo not compiled yet -- degrade gracefully
        _translator = gettext.NullTranslations()
```

Função procura o msgid equivalente ao que a variável global salvou e encontra a tradução, se não conseguir devolve o msgid mesmo;

```
def translate(msgid: str) -> str:
    """Translate *msgid* using the currently active language."""
    return _translator.gettext(msgid)
```

Inicializa o pacote de linguagem padrão quando importado;

```
set_language(_DEFAULT_LANGUAGE)
```

config.py

É utilizado para calcular conversão de pixels para centímetros em ambos os exercícios, para o `sit_and_reach.py` ele faz a medição dos ângulos para calibração, postura para medição, limites do lado oposto do lado do exercício e para controle de tempo e médias, para `back_scratch.py` limite de medida, tempo de pose, delay para reset ao perder pose, correção de erro;

```
# --- Pixel to CM conversion ---
# Approximate ratio at 1 meter distance
#SIT_AND_REACH_PIXEL_TO_CM = 0.533333
SIT_AND_REACH_PIXEL_TO_CM = 0.15
BACK_SCRATCH_PIXEL_TO_CM = 0.21

# --- Sit and Reach (SAR): angle boundaries ---
# Calibration phase
SAR_CALIB_ELBOW_MIN = 20
SAR_CALIB_ELBOW_MAX = 120
SAR_CALIB_HIP_MIN = 120
SAR_CALIB_HIP_MAX = 160
SAR_CALIB_KNEE_MIN = 140
SAR_CALIB_KNEE_MAX = 180

# Posture / measurement phase
SAR_POSTURE_ELBOW_MIN = 155
SAR_POSTURE_ELBOW_MAX = 180
SAR_POSTURE_HIP_MIN = 60
SAR_POSTURE_HIP_MAX = 150
SAR_POSTURE_KNEE_MIN = 140
SAR_POSTURE_KNEE_MAX = 180

# Opposite-side limits (shared between phases)
SAR_OPP_ELBOW_MIN = 155
SAR_OPP_ELBOW_MAX = 180
SAR_OPP_KNEE_MIN = 80
SAR_OPP_KNEE_MAX = 150

# --- Sit and Reach: timing & averaging ---
SAR_CALIBRATION_DURATION = 5 # seconds to hold calibration pose
SAR_POSE_DURATION = 5 # seconds to hold measurement pose
SAR_AVERAGE_OVER = 6 # rolling window for distance smoothing
SAR_ERROR_RIGHT = 4.9 # systematic error correction (cm) for
right side; base SAR_ERROR was 2.9
SAR_ERROR_LEFT = 2.9 # systematic error correction (cm) for
left side
SAR_SIGN_THRESHOLD = 1.0 # min distance (cm) beyond foot to
invert sign; avoids noise flipping positive/negative
```

```
# --- Back Scratch (BS): thresholds ---
BS_DISTANCE_THRESHOLD      = 33    # cm -- hands must be closer than this
BS_POSE_HELD_DURATION      = 3     # seconds to hold final pose
BS_POSE_NO_HELD_DURATION   = 1.5   # seconds before timer resets on lost
detection
BS_ERROR                    = 1.91  # systematic error correction (cm)

# --- MediaPipe pose landmark indices (for reference) ---
# 11=L_shoulder  12=R_shoulder  13=L_elbow  14=R_elbow
# 15=L_wrist     16=R_wrist     19=L_index  20=R_index
# 23=L_hip       24=R_hip       25=L_knee   26=R_knee
# 27=L_ankle     28=R_ankle     29=L_heel   30=R_heel
# 31=L_foot_idx  32=R_foot_idx
```

ui/draw.py

É utilizado para fazer os desenhos em toda a UI de forma padronizada, utiliza PIL para formatação de acentos e caracteres especiais e OpenCV para desenhar as formas geométricas.

Realiza as importações, importa as cores padronizadas de theme.py;

```
import unicodedata
import cv2
import numpy as np
from PIL import Image, ImageDraw, ImageFont
from typing import Optional

from ui.theme import (
    BG, DARK_BLUE, HEADER_BG, HEADER_TEXT, BTN_BLUE, BTN_TEXT,
    W, H, BORDER_INSET, HEADER_H, INSTITUTION,
)
```

_FONT_CACHE = {} é um dicionário global com as fontes PIL carregadas para não ler arial.ttf a cada frame, **_get_font_size()** é uma função para carregar arial.ttf/arialbd.ttf no tamanho pedido e guarda em cache;

```
_FONT_CACHE = {}

def _get_font_size(font_size, is_bold=False):
    """Return a PIL font for a given point size, with optional bold."""
    key = (font_size, is_bold)
    if key in _FONT_CACHE:
        return _FONT_CACHE[key]
    font_path = r"C:\Windows\Fonts\arialbd.ttf" if is_bold else
r"C:\Windows\Fonts\arial.ttf"
    try:
```

```

        font = ImageFont.truetype(font_path, font_size)
    except Exception:
        font = ImageFont.load_default()
    _FONT_CACHE[key] = font
    return font

```

Função desenha o texto com normalização NFC e converte BGR para RGB, é utilizada em todas as telas;

```

def put_text(img, text, pos, font_size=22, color=(0, 0, 0),
is_bold=False):
    """Draw UTF-8 text onto *img* in-place and return the modified
    image."""
    text = unicodedata.normalize('NFC', str(text))
    font = _get_font_size(font_size, is_bold)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(img_rgb)
    color_rgb = (color[2], color[1], color[0])
    ImageDraw.Draw(pil_img).text(pos, text, font=font, fill=color_rgb)
    img[:] = cv2.cvtColor(np.asarray(pil_img), cv2.COLOR_RGB2BGR)
    return img

```

Mede largura e altura do texto para o PIL, é utilizado quando um texto é preciso ser centralizado;

```

def measure_text(text, font_size=22, is_bold=False):
    """Helper to dynamically calculate text width and height for PIL
    rendering."""
    text = unicodedata.normalize('NFC', str(text))
    font = _get_font_size(font_size, is_bold)
    try:
        bbox = font.getmask(text).getbbox()
        if bbox:
            return bbox[2] - bbox[0], bbox[3] - bbox[1]
    except Exception:
        pass
    return len(text) * (font_size // 2), font_size

```

Desenha vários textos em uma única sessão PIL com uma lista de (texto, pos, size, cor, bold) por uma questão de eficiência na tela de `sit_and_reach.py`;

```

def put_text_multi(img, items):
    """Draw multiple texts in a single PIL session for efficiency.

    *items* is a list of (text, pos, font_size, color, is_bold) tuples.
    Returns the modified image.
    """
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    pil_img = Image.fromarray(img_rgb)
    draw = ImageDraw.Draw(pil_img)

```

```

for text, pos, font_size, color, is_bold in items:
    text = unicodedata.normalize('NFC', str(text))
    font = _get_font_size(font_size, is_bold)
    color_rgb = (color[2], color[1], color[0])
    draw.text(pos, text, font=font, fill=color_rgb)
return cv2.cvtColor(np.asarray(pil_img), cv2.COLOR_RGB2BGR)

```

Função cria um `ndarray` com preenchido com a cor BG, serve de background para `menu.py`, `exercise_intro.py` e `view_data.py`;

```

def blank_canvas() -> np.ndarray:
    """Return a fresh WxH canvas filled with the background colour."""
    img = np.zeros((H, W, 3), dtype=np.uint8)
    img[:] = BG
    return img

```

Função faz o retângulo azul que serve como borda para `draw_title_page()` e `menu.py`;

```

def draw_outer_border(img: np.ndarray):
    """Draw the thin dark-blue outer border."""
    cv2.rectangle(img,
                  (BORDER_INSET, BORDER_INSET),
                  (W - BORDER_INSET, H - BORDER_INSET),
                  DARK_BLUE, 2)

```

Função cria o texto “IPBeja” no canto superior direito das telas, utilizado em `draw_title_page()` e `menu.py`;

```

def draw_institution(img: np.ndarray, extra: str = ""):
    """
    Draw the institution label top-right.
    *extra* is appended after a ' / ' separator (e.g. operator name).
    """
    label = f"{INSTITUTION} / {extra}" if extra else INSTITUTION
    tw, th = measure_text(label, 15)
    put_text(img, label, (W - tw - 16, 30 - th), font_size=15,
              color=DARK_BLUE)

```

Função faz a barra azul no topo da tela com nome de exercício à esquerda, qual o lado ao centro e qual a repetição à direita, utilizada em `forms.py`;

```

def draw_header(img: np.ndarray,
                exercise: str,
                side: str,
                rep_current: int,
                rep_total: int):
    """Draw the full-width coloured top bar seen in exercise pages."""
    from locale_setup import translate
    cv2.rectangle(img, (0, 0), (W, HEADER_H), HEADER_BG, -1)

    _trash, text_h = measure_text("A", 24, is_bold=True)

```

```

text_y = max(0, (HEADER_H - text_h) // 2)

img[:] = put_text(img, exercise.upper(), (20, text_y),
                  font_size=24, color=tuple(HEADER_TEXT),
is_bold=True)

tw, _trash = measure_text(side, 24, is_bold=True)
img[:] = put_text(img, side, ((W - tw) // 2, text_y),
                  font_size=24, color=tuple(HEADER_TEXT),
is_bold=True)

rep_label = f"{translate('repetition_label')}"
{rep_current}/{rep_total}"
tw, _trash = measure_text(rep_label, 24, is_bold=True)
img[:] = put_text(img, rep_label, (W - tw - 20, text_y),
                  font_size=24, color=tuple(HEADER_TEXT),
is_bold=True)

```

Função desenha a moldura, as linhas decorativas e o título centralizado, utilizado em `menu.py`;

```

def draw_title_page(img: np.ndarray, title: str, institution_extra: str
= ""):
    """
    Draw outer border + decorative lines flanking the title +
    institution label.
    Matches the 'Menu Principal' / 'Sentar e Alcançar' / 'Próximo'
    pages.
    """
    draw_outer_border(img)
    draw_institution(img, institution_extra)

    tw, th = measure_text(title, 43)
    tx = (W - tw) // 2
    ty = 80 - th // 2

    line_y = ty + th // 2 + 5
    margin = 30
    line_x1 = BORDER_INSET + margin
    line_x2r = tx - 30
    line_x1r = tx + tw + 30
    line_x2 = W - BORDER_INSET - margin

    cv2.line(img, (line_x1, line_y), (line_x2r, line_y), DARK_BLUE, 2)
    cv2.line(img, (line_x1r, line_y), (line_x2, line_y), DARK_BLUE, 2)

    put_text(img, title, (tx, ty), font_size=43, color=DARK_BLUE)

```

Função desenha os botões com o texto centralizado que muda de cor se o mouse estiver em cima, utilizado em `menu.py`;

```
def draw_button(img: np.ndarray,
                label: str,
                x: int, y: int, w: int, h: int,
                hovered: bool = False):
    """Draw a filled rectangle button with centred label."""
    color = (110, 40, 40) if hovered else BTN_BLUE
    cv2.rectangle(img, (x, y), (x + w, y + h), color, -1)

    tw, th = measure_text(label, 20)
    tx = x + (w - tw) // 2
    ty = y + (h - th) // 2
    put_text(img, label, (tx, ty), font_size=20, color=BTN_TEXT)
```

Função desenha o cartão branco com borda escura para os formulários;

```
def draw_card(img: np.ndarray,
              x: int, y: int, w: int, h: int):
    """Draw a white card with dark-blue border."""
    cv2.rectangle(img, (x, y), (x + w, y + h), (255, 255, 255), -1)
    cv2.rectangle(img, (x, y), (x + w, y + h), DARK_BLUE, 2)
```

Função desenha o cabeçalho azul dentro do cartão já criado, com texto em branco centrado, utilizado em `forms.py`;

```
def draw_card_title(img: np.ndarray, title: str,
                    x: int, y: int, w: int, h: int = 52):
    """Draw filled header inside a card (e.g. 'CADASTRO')."""
    cv2.rectangle(img, (x, y), (x + w, y + h), BTN_BLUE, -1)
    tw, _ = measure_text(title, 28, is_bold=True)
    tx = x + (w - tw) // 2
    img[:] = put_text(img, title, (tx, y + 12), font_size=28,
                      color=(255, 255, 255), is_bold=True)

    return img
```

Função desenha os círculos de progresso, completo X verde, atual bola amarela e pendentes bolas azuis, utilizada em `exercise_intro.py`;

```
def draw_rep_circles(img: np.ndarray,
                     cx: int, cy: int,
                     total: int,
                     done: int,
                     radius: int = 38,
                     current: Optional[int] = None):
    if current is None:
        current = done

    gap = radius * 2 + 24
    total_w = total * (radius * 2) + (total - 1) * 24
```



```

start_x = cx - total_w // 2 + radius

for i in range(total):
    cx_i = start_x + i * gap
    if i < done:
        cv2.circle(img, (cx_i, cy), radius, BTN_BLUE, -1)
        d = int(radius * 0.45)
        cv2.line(img, (cx_i - d, cy - d), (cx_i + d, cy + d), (0,
255, 0), 3)
        cv2.line(img, (cx_i + d, cy - d), (cx_i - d, cy + d), (0,
255, 0), 3)
    elif i == current:
        cv2.circle(img, (cx_i, cy), radius, (0, 255, 255), -1)
        cv2.circle(img, (cx_i, cy), radius, BTN_BLUE, 4)
    else:
        cv2.circle(img, (cx_i, cy), radius, BTN_BLUE, -1)

```

ui/exercise_intro.py

Cria a tela mostrada antes de cada repetição para auxiliar o paciente e o utilizador, contém nome do exercício, círculos de progresso e dica para iniciar.

Realiza os imports;

```

import cv2
from utils import WindowManager
from locale_setup import translate
from ui.draw import (draw_rep_circles, blank_canvas,
                    put_text, measure_text)
from ui.theme import (
    W, H, BG, BORDER_INSET, BTN_BLUE, BTN_TEXT, DARK_BLUE, INSTITUTION,
)

```

Função desenha os retângulos de texto, os textos centralizados e os círculos de progresso com suas características;

```

def _draw_row(img, label, cx, cy, total, done, current=-1, radius=48):
    """Draw label button + circles for one side row."""
    btn_w, btn_h = 460, 64
    bx = cx - btn_w // 2
    by = cy - btn_h // 2
    cv2.rectangle(img, (bx, by), (bx + btn_w, by + btn_h), BTN_BLUE, -1)

    tw, th = measure_text(label, 36)
    put_text(img, label, (bx + (btn_w - tw) // 2, by + (btn_h - th) //
2), font_size=36, color=BTN_TEXT)

    circle_y = cy + btn_h // 2 + radius + 20

```

```
draw_rep_circles(img, cx, circle_y, total, done, radius, current)
```

Função recebe o nome do exercício, a repetição atual e o callback para abortar a sessão, calcula quantas repetições foram feitas, qual a repetição atual, qual o próximo exercício, o título, e a centralização das linhas do retângulo externo. No loop principal a janela é criada, o canvas é criado, a moldura é desenhada, o “IPBeja” é desenhado, o título é centralizado com fundo, desenha as formas (retângulo e 2 círculos) para cada lado, desenha o texto de rodapé e a barra inferior;

```
def show_exercise_intro(exercise_name, rep, finish_cb):
    right_done = min(rep, 2)
    left_done = max(rep - 2, 0)
    right_current = rep if rep < 2 else -1
    left_current = (rep - 2) if rep >= 2 else -1
    is_next = rep > 0
    title = translate("Next") if is_next else exercise_name

    row_gap = 180
    start_y = (H - row_gap * 2) // 2 - 20
    cx = W // 2

    right_label = f"{translate('right_side_label')} x2"
    left_label = f"{translate('left_side_label')} x2"

    wm = WindowManager(exercise_name, finish_cb, delay=16, size=(W, H))
    while not wm.should_close:
        img = blank_canvas()

        # Outer border with top edge at Y=80 (como o menu)
        cv2.rectangle(img,
                      (BORDER_INSET, 80),
                      (W - BORDER_INSET, H - BORDER_INSET),
                      DARK_BLUE, 2)

        # Institution Label -- right-aligned with right border
        inst_tw, inst_th = measure_text(INSTITUTION, 15)
        put_text(img, INSTITUTION, (W - BORDER_INSET - inst_tw, 70 -
inst_th),
                font_size=15, color=DARK_BLUE)

        # Title centered on the top border line (Y=80)
        tw, th = measure_text(title, 43)
        tx = (W - tw) // 2
        ty = 80 - th // 2
        pad = 20
        cv2.rectangle(img,
                      (tx - pad, ty - pad),
```

```

        (tx + tw + pad, ty + th + pad),
        BG, -1)
    put_text(img, title, (tx, ty), font_size=43, color=DARK_BLUE)

    # Right side row
    _draw_row(img, right_label, cx, start_y + 50,
              total=2, done=right_done, current=right_current)

    # Left side row
    _draw_row(img, left_label, cx, start_y + 80 + row_gap,
              total=2, done=left_done, current=left_current)

    # Footer hint
    hint = translate("Press ENTER to begin") if not is_next else
    translate("Press ENTER to continue")
    tw, th = measure_text(hint, 31)
    put_text(img, hint, ((W - tw) // 2, H - 75 - th), font_size=31,
    color=DARK_BLUE)

    # Bottom bar -- exercise name / side / rep counter
    side_label = translate("right_side_label") if rep < 2 else
    translate("left_side_label")
    rep_label = f"{translate('repetition_label')} {(rep % 2) +
    1}/2"

    _, ex_th = measure_text(exercise_name.upper(), 14)
    put_text(img, exercise_name.upper(), (50, H - 20 - ex_th),
    font_size=14, color=DARK_BLUE)

    tw, th = measure_text(side_label, 14)
    put_text(img, side_label, ((W - tw) // 2, H - 20 - th),
    font_size=14, color=DARK_BLUE)

    tw, th = measure_text(rep_label, 14)
    put_text(img, rep_label, (W - tw - 50, H - 20 - th),
    font_size=14, color=DARK_BLUE)

    wm.show(img)

    key = wm.poll()
    if key == "close":
        finish_cb()
        return
    elif key in (13, 10):
        wm.close()
        return

```

ui/forms.py

Faz os formulários utilizados em ambos os exercícios e centraliza-os, é utilizado antes do primeiro exercício para o registo, após fazer uma repetição faz a tela de medição real, então mostra a tela de distância real vs automática e erro e por último após as 4 repetições mostra um sumário com todos os valores.

Realiza as importações;

```
import cv2
import numpy as np
from locale_setup import translate
from ui.theme import (
    BG, DARK_BLUE, BTN_BLUE, BTN_TEXT, W, H,
)
from ui.draw import (draw_header, draw_card, draw_card_title,
                     put_text, measure_text)
from utils import WindowManager
```

Função para criar o canvas com fundo BG e faz o header, para criar a tela base antes dos overlays;

```
def _make_base(exercise, side, rep_current, rep_total):
    """Return a fresh canvas with background and header."""
    img = np.zeros((H, W, 3), dtype=np.uint8)
    img[:] = BG
    draw_header(img, exercise, side, rep_current, rep_total)
    return img
```

Função para criar a tela de registo, com os 4 campos: idade, altura, peso e género. Faz o card da janela, o título, as labels e caixas, possibilidade de input, continuar com Enter, sair com ESC;

```
def show_register_screen_styled(exercise, side, rep_current, rep_total,
                                finish_cb):
    """
    Styled registration form with card centered inside a themed
    background frame.
    """
    try:
        fields = [translate("Age"), translate("Height (cm)"),
                  translate("Weight (kg)"), translate("Gender (M/F)")]
        values = ["", "", "", ""]
        active_field = -1

        # Definição das dimensões da Janela e do Card Interno (Alinhado
        com o Mockup)
        win_w, win_h = 530, 570
        card_x, card_y = 0, 0
        card_w = win_w
```

```

card_h = win_h

field_x = card_x + 30
field_w = card_w - 60
field_h = 44
title_h = 58

fields_start = card_y + title_h + 20
block_gap = 105 # Distribuição vertical equilibrada

positions = []
label_y_positions = []

for i in range(len(fields)):
    y_start = fields_start + i * block_gap
    label_y_positions.append(y_start)

    x1 = field_x
    y1 = y_start + 32
    x2 = field_x + field_w
    y2 = y1 + field_h
    positions.append((x1, y1, x2, y2))

def _mouse_cb(event, x, y, flags, param):
    nonlocal active_field
    if event == cv2.EVENT_LBUTTONDOWN:
        active_field = -1
        for i, (x1, y1, x2, y2) in enumerate(positions):
            if x1 <= x <= x2 and y1 <= y <= y2:
                active_field = i
                break

wm = WindowManager("Register", finish_cb, delay=10, size=(win_w,
win_h), on_mouse=_mouse_cb)
while not wm.should_close:
    # Fundo principal com a cor do Tema (O azul claro que
emoldura o ecrã)
    img = np.zeros((win_h, win_w, 3), dtype=np.uint8)
    img[:] = BG

    # 1. Desenha apenas o PREENCHIMENTO do Retângulo interno
Branco (Card)
    cv2.rectangle(img, (card_x, card_y), (card_x + card_w,
card_y + card_h), (255, 255, 255), -1)

    # 2. Desenha a barra de título interior (CADASTRO)

```

```

        # Como o BG da janela é azul claro, o card fica destacado no
        centro sem tocar nas bordas
        img = draw_card_title(img, translate("Register").upper(),
        card_x, card_y, card_w, title_h)

        # Desenho das Labels e Caixas de Texto
        for i, (x1, y1, x2, y2) in enumerate(positions):
            # Renderização da Label em Negrito
            img = put_text(img, f"{fields[i]}:", (x1,
label_y_positions[i]), font_size=24, color=DARK_BLUE, is_bold=True)

            # Caixa de entrada de dados
            cv2.rectangle(img, (x1, y1), (x2, y2), (255, 255, 255),
-1)

            border = BTN_BLUE if i == active_field else DARK_BLUE
            cv2.rectangle(img, (x1, y1), (x2, y2), border, 2)

            # Texto digitado pelo utilizador
            img = put_text(img, values[i], (x1 + 12, y1 + 8),
font_size=22, color=(0, 0, 0))

        # Instrução inferior dentro da caixa
        hint_text = translate("enter_confirm")
        hint_font_size = 26
        text_w, text_h = measure_text(hint_text, hint_font_size)
        hint_x = (win_w - text_w) // 2
        hint_y = card_y + card_h - 25 - text_h

        img = put_text(img, hint_text, (hint_x, hint_y),
font_size=hint_font_size, color=tuple(DARK_BLUE), is_bold=False)

        wm.show(img)

        key = wm.poll()
        if key == "close":
            finish_cb()
            return
        elif key in (13, 10):
            if all(v.strip() for v in values):
                wm.close()
                return tuple(values)
        elif key == 9:
            active_field = (active_field + 1) % len(fields)
        elif active_field != -1:
            if key == 8:
                values[active_field] = values[active_field][:-1]

```

```

        elif 32 <= key <= 126:
            values[active_field] += chr(key)

    except Exception as e:
        import traceback
        print(f"[register] ERRO: {type(e).__name__}: {e}")
        traceback.print_exc()
        raise

```

Função para criar a tela para inserir a distância real, medida pelo utilizador, card com título, labels, caixa de texto, input, verificação de teclas válidas, ações para Enter, Backspace e ESC;

```

def show_real_distance_screen_styled(exercise, side, rep_current,
rep_total, finish_cb):
    """Styled real-distance input with header bar and white card."""
    entered = ""

    card_w, card_h = 580, 260
    title_h = 58

    win_h = card_h
    win_w = card_w

    card_x_local = 0
    card_y_local = 0
    field_x_local = card_x_local + 30
    field_w_local = card_w - 60
    field_y_local = card_y_local + title_h + 50

    field_h = 60

    wm = WindowManager("Real Measurement", finish_cb, delay=10,
size=(win_w, win_h))
    while not wm.should_close:
        img = np.zeros((win_h, win_w, 3), dtype=np.uint8)
        img[:] = BG

        draw_card(img, card_x_local, card_y_local, card_w, card_h)

        img = draw_card_title(img, translate("Real
Measurement").upper(), card_x_local, card_y_local, card_w, title_h)
        img = put_text(img, f"{translate('real_distance_label')} (cm):",
                        (field_x_local, card_y_local + title_h + 16),
                        font_size=20, color=tuple(DARK_BLUE), is_bold=True)

    # Input field

```

```

        cv2.rectangle(img, (field_x_local, field_y_local),
                       (field_x_local + field_w_local, field_y_local +
field_h),
                       (255, 255, 255), -1)
        cv2.rectangle(img, (field_x_local, field_y_local),
                       (field_x_local + field_w_local, field_y_local +
field_h),
                       DARK_BLUE, 2)
    if not entered:
        img = put_text(img, "+3.33 / -3.33",
                       (field_x_local + 12, field_y_local + 12),
                       font_size=24, color=(180, 190, 200))
    else:
        img = put_text(img, entered,
                       (field_x_local + 12, field_y_local + 12),
                       font_size=28, color=(0, 0, 200),
is_bold=True)

    # Hint
    _tc = translate("enter_confirm")
    _tw, _th = measure_text(_tc, 26)
    img = put_text(img, _tc,
                   ((win_w - _tw) // 2, field_y_local + field_h +
19),
                   font_size=26, color=tuple(DARK_BLUE))

    wm.show(img)

    key = wm.poll()
    if key == "close":
        finish_cb()
        return
    elif key in (13, 10) and entered:
        wm.close()
        return float(entered.replace(",", "."))
    elif key == 8:
        entered = entered[:-1]
    elif (48 <= key <= 57) or key in (44, 46, 43, 45):
        entered += chr(key)

```

Função para criar a tela de resultado da repetição, 3 blocos, Medida do Sistema, Medida Real e Erro, barra inferior, ação para Enter e ESC;

```

def show_repetition_result(exercise, side, rep_current, rep_total,
                           system_dist, real_dist, error, finish_cb):

    win_w = 700
    win_h = 520

```



```

card_w = win_w
card_h = win_h - 60
card_x = 0
card_y = 10

title_h = 58
block_h = 48
value_h = 48
block_gap = 24
content_x = card_x + 40
content_w = card_w - 80

sys_title_y = card_y + title_h + 20
sys_value_y = sys_title_y + block_h + 8
real_title_y = sys_value_y + value_h + block_gap
real_value_y = real_title_y + block_h + 8
error_title_y = real_value_y + value_h + block_gap

wm = WindowManager("Repetition Completed", finish_cb, delay=16,
size=(win_w, win_h))
while not wm.should_close:
    img = np.zeros((win_h, win_w, 3), dtype=np.uint8)
    img[:] = BG

    draw_card(img, card_x, card_y, card_w, card_h)
    img = draw_card_title(img, translate("Repetition
Completed").upper(), card_x, card_y, card_w, title_h)

    # System distance
    cv2.rectangle(img, (content_x, sys_title_y), (content_x +
content_w, sys_title_y + block_h), BTN_BLUE, -1)
    img = put_text(img, translate("System Measurement"), (content_x
+ 16, sys_title_y + 10), font_size=22, color=(255, 255, 255),
is_bold=True)

    cv2.rectangle(img, (content_x, sys_value_y), (content_x +
content_w, sys_value_y + value_h), (255, 255, 255), -1)
    cv2.rectangle(img, (content_x, sys_value_y), (content_x +
content_w, sys_value_y + value_h), DARK_BLUE, 2)
    img = put_text(img, f"{translate('system_distance_label')}:
{system_dist} cm",
                    (content_x + 16, sys_value_y + 10),
font_size=26, color=tuple(DARK_BLUE))

    # Real distance

```

```

        cv2.rectangle(img, (content_x, real_title_y), (content_x +
content_w, real_title_y + block_h), BTN_BLUE, -1)
        img = put_text(img, translate("Real Measurement Result"),
(content_x + 16, real_title_y + 10), font_size=22, color=(255, 255,
255), is_bold=True)

        cv2.rectangle(img, (content_x, real_value_y), (content_x +
content_w, real_value_y + value_h), (255, 255, 255), -1)
        cv2.rectangle(img, (content_x, real_value_y), (content_x +
content_w, real_value_y + value_h), DARK_BLUE, 2)
        img = put_text(img, f"{translate('real_distance_label')}:
{real_dist:.2f} cm",
                        (content_x + 16, real_value_y + 10),
font_size=26, color=tuple(DARK_BLUE))

        # Error section (filled header with value)
        cv2.rectangle(img, (content_x, error_title_y), (content_x +
content_w, error_title_y + block_h), BTN_BLUE, -1)
        img = put_text(img, f"{translate('Error')}: {error:.2f} cm",
                        (content_x + 16, error_title_y + 10),
font_size=22, color=(255, 255, 255), is_bold=True)

        _sc = translate("enter_esc")
        _tw, _th = measure_text(_sc, 26)
        hint_y = card_y + card_h - 10 - _th
        img = put_text(img, _sc, ((win_w - _tw) // 2, hint_y),
font_size=26, color=tuple(DARK_BLUE))

        font_size_bottom = 16
        _tw, text_h_bottom = measure_text("A", font_size_bottom)
        _space = win_h - (card_y + card_h)
        bottom_y = card_y + card_h + (_space - text_h_bottom) // 2

        rep_label = f"{translate('repetition_label')}
{rep_current}/{rep_total}"

        img = put_text(img, exercise.upper(), (20, bottom_y),
font_size=font_size_bottom, color=tuple(DARK_BLUE), is_bold=True)

        tw_side, _trash = measure_text(side, font_size_bottom)
        img = put_text(img, side, ((win_w - tw_side) // 2, bottom_y),
font_size=font_size_bottom, color=tuple(DARK_BLUE), is_bold=True)

        tw_rep, _trash = measure_text(rep_label, font_size_bottom)
        img = put_text(img, rep_label, (win_w - tw_rep - 20, bottom_y),
font_size=font_size_bottom, color=tuple(DARK_BLUE), is_bold=True)

```

```

wm.show(img)

key = wm.poll()
if key == "close":
    finish_cb()
    return
elif key in (13, 10):
    wm.close()
    return

```

Função para criar a tela de final do exercício com os resultados das 4 repetições, layout: 2 colunas e 2 linhas, seção Lado Direito e Lado Esquerdo, mostra Medida do Sistema, Medida Real e Erro para cada exercício, contém título e ações para ESC e Enter;

```

def show_exercise_final(exercise,
                        system_right_1, system_right_2,
                        system_left_1,  system_left_2,
                        real_right_1,   real_right_2,
                        real_left_1,    real_left_2,
                        error_right_1,  error_right_2,
                        error_left_1,   error_left_2,
                        finish_cb):

    wm = WindowManager("Exercise Completed", finish_cb, delay=16,
                       size=(W, H))

    card_x, card_y = 40, 40
    card_w, card_h = W - 80, H - 120
    title_h      = 58
    sec_h        = 48
    rep_h        = 28
    box_h        = 140
    gap          = 12
    col_w        = (card_w - 80) // 2
    col1_x       = card_x + 40
    col2_x       = col1_x + col_w + 20

    right_sec_y  = card_y + title_h + gap
    right_rep_y  = right_sec_y + sec_h + 4
    right_box_y  = right_rep_y + rep_h + 2
    left_sec_y   = right_box_y + box_h + gap + 4
    left_rep_y   = left_sec_y + sec_h + 4
    left_box_y   = left_rep_y + rep_h + 2

    while not wm.should_close:
        img = np.zeros((H, W, 3), dtype=np.uint8)

```

```

img[:] = BG

cv2.rectangle(img, (card_x, card_y), (card_x + card_w, card_y +
card_h), DARK_BLUE, 2)

img = put_text(img, "IPBeja", (W - 110, 12), font_size=18,
color=tuple(DARK_BLUE), is_bold=True)

title_text = translate("Exercise Completed")
title_font_size = 36
tw, th = measure_text(title_text, title_font_size, is_bold=True)
tx = (W - tw) // 2
ty = card_y - th // 2
pad = 14
cv2.rectangle(img, (tx - pad, ty - pad), (tx + tw + pad, ty + th
+ pad), BG, -1)
img = put_text(img, title_text, (tx, ty),
font_size=title_font_size, color=tuple(DARK_BLUE), is_bold=True)

# — Secção Lado Direito —
cv2.rectangle(img, (col1_x, right_sec_y), (col1_x + col_w * 2 +
20, right_sec_y + sec_h), BTN_BLUE, -1)
lbl_right = translate("right_side_label")
tw, th = measure_text(lbl_right, 24, is_bold=True)
img = put_text(img, lbl_right, (col1_x + (col_w * 2 + 20 - tw)
// 2, right_sec_y + (sec_h - th) // 2),
font_size=24, color=(255, 255, 255),
is_bold=True)

# Subtítulos repetições direito
for ci, label in enumerate([translate("rep_1_label"),
translate("rep_2_label")]):
    cx = col1_x if ci == 0 else col2_x
    tw, th = measure_text(label, 18, is_bold=True)
    img = put_text(img, label, (cx + (col_w - tw) // 2,
right_rep_y + (rep_h - th) // 2),
font_size=18, color=tuple(DARK_BLUE),
is_bold=True)

# Caixas valores direito
for ci, (sys_v, real_v, err_v) in enumerate([(system_right_1,
real_right_1, error_right_1), (system_right_2, real_right_2,
error_right_2)]):
    cx = col1_x if ci == 0 else col2_x
    box_y = right_box_y
    cv2.rectangle(img, (cx, box_y), (cx + col_w, box_y + box_h),

```

```

(255, 255, 255), -1)
    cv2.rectangle(img, (cx, box_y), (cx + col_w, box_y + box_h),
DARK_BLUE, 2)

    img = put_text(img, f"{translate('system_distance_label')}:
{sys_v} cm",
                    (cx + 16, box_y + 14), font_size=26,
color=tuple(DARK_BLUE))
    img = put_text(img, f"{translate('real_distance_label')}:
{real_v:.2f} cm",
                    (cx + 16, box_y + 48), font_size=26,
color=tuple(DARK_BLUE))
    img = put_text(img, f"{translate('Erro')}: {err_v:.2f} cm",
                    (cx + 16, box_y + 82), font_size=26,
color=tuple(DARK_BLUE))

    # — Seção Lado Esquerdo —
    cv2.rectangle(img, (col1_x, left_sec_y), (col1_x + col_w * 2 +
20, left_sec_y + sec_h), BTN_BLUE, -1)
    lbl_left = translate("left_side_label")
    tw, th = measure_text(lbl_left, 24, is_bold=True)
    img = put_text(img, lbl_left, (col1_x + (col_w * 2 + 20 - tw) //
2, left_sec_y + (sec_h - th) // 2),
                    font_size=24, color=(255, 255, 255),
is_bold=True)

    # Subtítulos repetições esquerdo
    for ci, label in enumerate([translate("rep_1_label"),
translate("rep_2_label")]):
        cx = col1_x if ci == 0 else col2_x
        tw, th = measure_text(label, 18, is_bold=True)
        img = put_text(img, label, (cx + (col_w - tw) // 2,
left_rep_y + (rep_h - th) // 2),
                            font_size=18, color=tuple(DARK_BLUE),
is_bold=True)

    # Caixas valores esquerdo
    for ci, (sys_v, real_v, err_v) in enumerate([(system_left_1,
real_left_1, error_left_1), (system_left_2, real_left_2,
error_left_2)]):
        cx = col1_x if ci == 0 else col2_x
        box_y = left_box_y
        cv2.rectangle(img, (cx, box_y), (cx + col_w, box_y + box_h),
(255, 255, 255), -1)
        cv2.rectangle(img, (cx, box_y), (cx + col_w, box_y + box_h),
DARK_BLUE, 2)

```

```

        img = put_text(img, f"{translate('system_distance_label')}:
{sys_v} cm",
                        (cx + 16, box_y + 14), font_size=26,
color=tuple(DARK_BLUE))
        img = put_text(img, f"{translate('real_distance_label')}:
{real_v:.2f} cm",
                        (cx + 16, box_y + 48), font_size=26,
color=tuple(DARK_BLUE))
        img = put_text(img, f"{translate('Erro')}: {err_v:.2f} cm",
                        (cx + 16, box_y + 82), font_size=26,
color=tuple(DARK_BLUE))

    ex_upper = exercise.upper()
    tw, th = measure_text(ex_upper, 20, is_bold=True)
    img = put_text(img, ex_upper, ((W - tw) // 2, card_y + card_h +
10), font_size=20, color=tuple(DARK_BLUE), is_bold=True)

    hint_exit = translate("esc_finish")
    tw, th = measure_text(hint_exit, 26)
    img = put_text(img, hint_exit, ((W - tw) // 2, card_y + card_h -
15 - th), font_size=26, color=tuple(DARK_BLUE))

    wm.show(img)

    key = wm.poll()
    if key == "close":
        finish_cb()
        break

wm.close()

```

ui/menu.py

Tela principal do sistema que tem as opções: automático, faz ambos os exercícios sendo sentar e alcançar primeiro, sentar e alcançar, faz somente esse exercício e volta para o menu, alcançar atrás das costas, faz somente esse exercício e volta para o menu, visualizar dados, gera gráficos de acordo com o que foi selecionado, e terminar sessão que encerra o programa.

Realiza os imports;

```

import cv2
import numpy as np
import os

import locale_setup

```

```

from locale_setup import translate
from ui.draw import blank_canvas, draw_button, put_text, measure_text
from ui.theme import W, H, BG, BORDER_INSET, DARK_BLUE, BTN_BLUE
from utils import set_app_icon, WindowManager

```

Realiza mapeamento dos diretórios da logo nos arquivos;

```

_BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
_LOGO_PATH = os.path.join(_BASE_DIR, "arquivos", "logo_final_green.png")

```

Faz as definições dos botões e salva a tradução, cria padrões para os mesmos;

```

_BUTTONS = [
    ("Automatic", "auto"),
    ("Sit and Reach", "sit_and_reach"),
    ("Back Scratch exercise name", "back_scratch"),
    ("Visualize Data", "view_data"),
    ("End Session", "quit"),
]

_BTN_W = 480
_BTN_H = 58
_BTN_GAP = 22

```

Função para posicionar os botões centralizando-os;

```

def _button_rects():
    """Return (x, y, w, h) for each button, vertically centred in the
    card."""
    total_h = len(_BUTTONS) * _BTN_H + (len(_BUTTONS) - 1) * _BTN_GAP
    start_y = (H - total_h) // 2 + 50
    x = (W - _BTN_W) // 2
    rects = []
    for i in range(len(_BUTTONS)):
        y = start_y + i * (_BTN_H + _BTN_GAP)
        rects.append((x, y, _BTN_W, _BTN_H))
    return rects

```

Função para desenhar as bandeiras para troca de idioma, primeira desenha bandeira portuguesa segunda desenha a inglesa;

```

def _draw_mini_pt_flag(img, x, y, w, h):
    """Draws a minimalist Portuguese flag combining basic geometric
    primitives."""
    # 1. Proporção oficial de fundo: 40% Verde, 60% Vermelho
    green_w = int(w * 0.4)
    cv2.rectangle(img, (x, y), (x + green_w, y + h), (0, 102, 0), -1)
    # Verde Clássico
    cv2.rectangle(img, (x + green_w, y), (x + w, y + h), (0, 0, 204),
    -1) # Vermelho Clássico

```

```

# 2. Centro da Esfera Armilar
cx, cy = x + green_w, y + h // 2
r_esfera = int(h * 0.3)

# Esfera Armilar (Círculo Amarelo -> formato BGR no OpenCV)
cv2.circle(img, (cx, cy), r_esfera, (20, 200, 220), -1)

# 3. Brasão em Branco (União de um Quadrado em cima com um Círculo
em baixo)
w_escudo = int(r_esfera * 1.1)
r_escudo = w_escudo // 2

y_topo = cy - int(r_esfera * 0.5)
y_meio = cy + int(r_esfera * 0.1) # Ponto exato onde o quadrado
termina e o círculo começa

# Parte superior do brasão (Retângulo/Quadrado Branco)
cv2.rectangle(img, (cx - r_escudo, y_topo), (cx + r_escudo, y_meio),
(255, 255, 255), -1)

# Parte inferior do brasão (Círculo Branco para arredondar a base)
cv2.circle(img, (cx, y_meio), r_escudo, (255, 255, 255), -1)

def _draw_mini_uk_flag(img, x, y, w, h):
    """Draws a mini Union Jack flag using OpenCV primitives."""
    cv2.rectangle(img, (x, y), (x + w, y + h), (128, 0, 0), -1)
    cv2.line(img, (x, y), (x + w, y + h), (255, 255, 255), 2)
    cv2.line(img, (x + w, y), (x, y + h), (255, 255, 255), 2)
    cv2.line(img, (x, y), (x + w, y + h), (0, 0, 220), 1)
    cv2.line(img, (x + w, y), (x, y + h), (0, 0, 220), 1)
    cv2.line(img, (x + w//2, y), (x + w//2, y + h), (255, 255, 255), 4)
    cv2.line(img, (x, y + h//2), (x + w, y + h//2), (255, 255, 255), 4)
    cv2.line(img, (x + w//2, y), (x + w//2, y + h), (0, 0, 220), 2)
    cv2.line(img, (x, y + h//2), (x + w, y + h//2), (0, 0, 220), 2)

```

Função da criação do menu, inicia com português como língua padrão, carrega a logotipo ao lado do título, verifica qual a bandeira atual mudando a língua, cria os elementos visuais da tela, centraliza os itens principais, desenha o título, renderiza os botões e as bandeiras e retorna a seleção escolhida;

```

def show_main_menu() -> str:
    """
    Display the main menu and return the selected action key.
    """
    rects = _button_rects()
    selected = None

```



```

hover      = None

current_lang = "pt_PT"

# Geometria da Mini-Bandeira no Canto Inferior Esquerdo
flag_x, flag_y = 65, H - 95
flag_w, flag_h = 60, 40
hover_flag     = False

# Carrega o Logótipo em modo UNCHANGED
logo_img = cv2.imread(_LOGO_PATH, cv2.IMREAD_UNCHANGED)

def _mouse(event, x, y, flags, param):
    nonlocal selected, hover, current_lang, hover_flag
    hover = None
    hover_flag = False

    for i, (bx, by, bw, bh) in enumerate(rects):
        if bx <= x <= bx + bw and by <= y <= by + bh:
            hover = i
            if event == cv2.EVENT_LBUTTONDOWN:
                selected = _BUTTONS[i][1]
                return

    if flag_x <= x <= flag_x + flag_w and flag_y <= y <= flag_y +
flag_h:
        hover_flag = True
        if event == cv2.EVENT_LBUTTONDOWN:
            current_lang = "en_US" if current_lang == "pt_PT" else
"pt_PT"

            locale_setup.set_language(current_lang)

    wm = WindowManager("Main Menu", size=(W, H), delay=16,
on_mouse=_mouse)
    set_app_icon(wm.winname)

    while selected is None:
        img = blank_canvas()

        top_border_y = 80

        # Simple outer rectangle border (top edge lowered)
        cv2.rectangle(img, (BORDER_INSET, top_border_y),
            (W - BORDER_INSET, H - BORDER_INSET), DARK_BLUE,

```

2)

```

    # Institution Label -- above the top border, right-aligned
    inst_tw, inst_th = measure_text("IPBeja", 18)
    img = put_text(img, "IPBeja", (W - BORDER_INSET - inst_tw,
top_border_y - inst_th - 6),
                    font_size=18, color=tuple(DARK_BLUE))

    # 1. PROCESSAMENTO DE MEDIDAS E CENTRALIZAÇÃO DO BLOCO [LOGO +
    TEXTO]
    title_text = "CAPACITA"
    font_size = 60
    text_w, text_h = measure_text(title_text, font_size,
is_bold=True)

    gap_logo_text = 5
    title_y = top_border_y - text_h // 2

    if logo_img is not None:
        logo_height_target = 100

        h_logo, w_logo = logo_img.shape[:2]
        new_w_logo = int(logo_height_target * (w_logo / h_logo))
        logo_resized = cv2.resize(logo_img, (new_w_logo,
logo_height_target))

        logo_draw_y = top_border_y - logo_height_target // 2

        start_x = (W - 480) // 2

        pad_box = 14
        block_top = min(logo_draw_y, title_y)
        block_h = max(logo_height_target, text_h)
        cv2.rectangle(img,
                      (start_x - pad_box, block_top - pad_box),
                      (start_x + 480 + pad_box, block_top + block_h
+ pad_box),
                      BG, -1)

        try:
            roi = img[logo_draw_y:logo_draw_y+logo_height_target,
start_x:start_x+new_w_logo]

            if logo_resized.shape[2] == 4:
                alpha = logo_resized[:, :, 3] / 255.0
                alpha = np.expand_dims(alpha, axis=2)
                blended = logo_resized[:, :, :3] * alpha + roi *

```

```

(1.0 - alpha)
        img[logo_draw_y:logo_draw_y+logo_height_target,
start_x:start_x+new_w_logo] = blended.astype(np.uint8)
    else:
        logo_gray = cv2.cvtColor(logo_resized,
cv2.COLOR_BGR2GRAY)
        thresh_val, mask = cv2.threshold(logo_gray, 15, 255,
cv2.THRESH_BINARY)
        mask_inv = cv2.bitwise_not(mask)
        img_bg = cv2.bitwise_and(roi, roi, mask=mask_inv)
        logo_fg = cv2.bitwise_and(logo_resized,
logo_resized, mask=mask)
        img[logo_draw_y:logo_draw_y+logo_height_target,
start_x:start_x+new_w_logo] = cv2.add(img_bg, logo_fg)
    except Exception as e:
        print(f"Erro ao renderizar logo: {e}")

    text_final_x = start_x + new_w_logo + gap_logo_text
else:
    text_final_x = (W - 480) // 2

    # Desenha o texto "CAPACITA" em Negrito
    img = put_text(img, title_text, (text_final_x, title_y),
font_size=font_size, color=tuple(DARK_BLUE), is_bold=True)

# 2. RENDERIZAÇÃO DOS BOTÕES DO MENU
for i, (msgid, _action) in enumerate(_BUTTONS):
    bx, by, bw, bh = rects[i]
    draw_button(img, "", bx, by, bw, bh, hovered=(hover == i))

    texto_traduzido = translate(msgid)
    tw, th = measure_text(texto_traduzido, 22)
    text_x = (W - tw) // 2
    text_y = by + (bh - th) // 2
    img = put_text(img, texto_traduzido, (text_x, text_y),
font_size=22, color=(255, 255, 255))

# 3. RENDERIZAÇÃO DA BANDEIRA INVERSA (Alternância de Idioma)
if current_lang == "pt_PT":
    _draw_mini_uk_flag(img, flag_x, flag_y, flag_w, flag_h)
else:
    _draw_mini_pt_flag(img, flag_x, flag_y, flag_w, flag_h)

if hover_flag:
    cv2.rectangle(img, (flag_x - 3, flag_y - 3), (flag_x +
flag_w + 3, flag_y + flag_h + 3), (200, 210, 230), 2)

```

```

        else:
            cv2.rectangle(img, (flag_x, flag_y), (flag_x + flag_w,
flag_y + flag_h), DARK_BLUE, 1)

            wm.show(img)
            key = wm.poll()
            if key == "close":
                selected = "quit"

wm.close()
return selected

```

ui/theme.py

Para ser importada na criação das telas para padronizar o design;

```

BG = (248, 245, 240)    # #F8F5F0 -- light blue-grey background
DARK_BLUE   = (142,  46,  46)    # #2E2E8E -- primary dark blue (borders,
text)
BTN_BLUE    = (139,  56,  56)    # #383894 -- button fill
BTN_TEXT    = (255, 255, 255)    # white    -- button label
HEADER_BG   = (139,  56,  56)    # #383894 -- top bar fill
HEADER_TEXT = (255, 255, 255)    # white

# --- Layout ---
W, H        = 1280, 720    # window size
BORDER_INSET = 40          # outer border inset (px)
HEADER_H     = 60          # top bar height (px)
INSTITUTION  = "IPBeja"

```

ui/view_data.py

Mostra gráficos interativos de ambos os exercícios ou separadamente, opções de filtro incluem: gênero, lado e intervalo de repetições.

Faz as importações necessárias;

```

import cv2
import numpy as np
import pandas as pd
import os

# Configuração absoluta do backend em memória do Matplotlib
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

```

```

from locale_setup import translate
from ui.draw import blank_canvas, put_text, measure_text
from ui.theme import W, H, BORDER_INSET, DARK_BLUE, BTN_BLUE, BG
from utils import WindowManager

```

Faz mapeamento dos diretórios das tabelas para criar os gráficos;

```

_BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
_PATH_SR = os.path.join(_BASE_DIR, "arquivos", "tabelas_utentes",
"sit_and_reach_2_utentes.xlsx")
_PATH_BS = os.path.join(_BASE_DIR, "arquivos", "tabelas_utentes",
"back_scratch_utentes.xlsx")

```

Função gera o gráfico com matplotlib de todas as execuções inicialmente, e depois quando for clicado calcular ele gera novamente de acordo com as escolhas;

```

def _generate_processed_chart(df):
    """Gera o Line Chart comparativo das repetições."""
    plt.rcParamsdefaults()
    try:
        if df.empty: return None

        df_chart = df.reset_index(drop=True)

        fig = plt.figure(figsize=(9.2, 3.4), dpi=100)
        ax = fig.add_subplot(111)

        real_distance_color = "#FF0000"
        calculated_distance_color = '#142E8B'

        if 'Real distance' in df_chart.columns and 'Calculated distance'
in df_chart.columns:
            ax.plot(df_chart.index + 1, df_chart['Real distance'],
marker='o', linewidth=2, color=real_distance_color,
label=translate('real_distance_label'))
            ax.plot(df_chart.index + 1, df_chart['Calculated distance'],
marker='s', linewidth=2, color=calculated_distance_color,
label=translate('system_distance_label'))

            ax.set_title(translate("chart_title"), fontsize=11,
fontweight='bold', color=calculated_distance_color, pad=6)
            ax.set_xlabel(translate("chart_xlabel"), fontsize=9,
color=calculated_distance_color)
            ax.set_ylabel("cm", fontsize=9, color=calculated_distance_color)

            ax.grid(True, linestyle='--', alpha=0.5, color='#CBD5E1')
            ax.spines['top'].set_visible(False)
            ax.spines['right'].set_visible(False)

```

```

        ax.legend(loc='upper right', frameon=True, facecolor='#F8FAFC',
edgecolor=real_distance_color)

    plt.tight_layout()

    fig.canvas.draw()
    img_rgba = np.array(fig.canvas.buffer_rgba())
    img_bgr = cv2.cvtColor(img_rgba, cv2.COLOR_RGBA2BGR)

    plt.clf()
    plt.close(fig)
    return img_bgr
except Exception as e:
    print(f"Erro ao gerar gráfico: {e}")
    return None

```

Função define os checkboxes, e faz a marcação quando selecionados;

```

def _draw_chk(canvas, x, y, checked, label):
    cv2.rectangle(canvas, (x, y), (x + 24, y + 24), tuple(DARK_BLUE), 2)
    if checked:
        cv2.rectangle(canvas, (x + 5, y + 5), (x + 19, y + 19),
tuple(DARK_BLUE), -1)
        return put_text(canvas, label, (x + 36, y + 3), font_size=16,
color=tuple(DARK_BLUE))

```

Função carrega os dois arquivos excel, os sliders, os checkboxes, as barras bicolores e as legendas do gráfico, o botão de calcular e calcula, gera o gráfico;

```

def show_data_visualization():
    wm = WindowManager("Visualize Data", size=(W, H), delay=20,
on_mouse=_mouse_callback)

    # Dicionário de Estados das Checkboxes
    opts = {
        "exe_sr": True, "exe_bs": True,
        "gen_f": True, "gen_m": True,
        "side_esq": True, "side_dir": True
    }

    # — Per-exercise data loading —
    sr_full = pd.DataFrame()
    if os.path.exists(_PATH_SR):
        try:
            sr_full = pd.read_excel(_PATH_SR)
        except Exception as e: print(f"Erro SR: {e}")
    bs_full = pd.DataFrame()
    if os.path.exists(_PATH_BS):
        try:

```

```

        bs_full = pd.read_excel(_PATH_BS)
    except Exception as e: print(f"Erro BS: {e}")

    sr_total = len(sr_full)
    bs_total = len(bs_full)
    sr_min = 0
    sr_max = max(0, sr_total - 1)
    bs_min = 0
    bs_max = max(0, bs_total - 1)
    dragging = None # ("sr", "min"), ("sr", "max"), ("bs", "min"),
    ("bs", "max") or None

    # — Filtering helpers —
    def _apply_filters(df):
        if df.empty:
            return df
        if 'Gender' in df.columns:
            if opts["gen_f"] and not opts["gen_m"]:
                df =
df[df['Gender'].astype(str).str.strip().str.upper().str.startswith('F')]
            elif opts["gen_m"] and not opts["gen_f"]:
                df =
df[df['Gender'].astype(str).str.strip().str.upper().str.startswith('M')]
        if 'Side' in df.columns:
            if opts["side_dir"] and not opts["side_esq"]:
                df = df[df['Side'].astype(str).str.lower() == 'right']
            elif opts["side_esq"] and not opts["side_dir"]:
                df = df[df['Side'].astype(str).str.lower() == 'left']
        return df

    def _build_filtered_data():
        df_list = []
        if opts["exe_sr"] and not sr_full.empty:
            d = sr_full.iloc[sr_min:sr_max+1].copy()
            d['Origem_Ex'] = "Sit & Reach"
            df_list.append(d)
        if opts["exe_bs"] and not bs_full.empty:
            d = bs_full.iloc[bs_min:bs_max+1].copy()
            d['Origem_Ex'] = "Back Scratch"
            df_list.append(d)
        if not df_list:
            return pd.DataFrame()
        df = pd.concat(df_list, ignore_index=True)
        return _apply_filters(df)

    df_current = _build_filtered_data()

```

```

current_chart = _generate_processed_chart(df_current)

# — Slider constants —
SLIDER_LEFT  = 370
SLIDER_RIGHT = 600
SLIDER_H     = 8
HANDLE_R     = 8
SR_SLIDER_Y  = 172
BS_SLIDER_Y  = 222

def _draw_slider(canvas, idx_to_px, cur_min, cur_max, total,
slider_y):
    if total <= 1:
        return canvas
    min_px = idx_to_px(cur_min)
    max_px = idx_to_px(cur_max)
    cv2.rectangle(canvas, (SLIDER_LEFT, slider_y - SLIDER_H//2),
(min_px, slider_y + SLIDER_H//2), tuple(DARK_BLUE), -1)
    cv2.rectangle(canvas, (min_px, slider_y - SLIDER_H//2), (max_px,
slider_y + SLIDER_H//2), tuple(BTN_BLUE), -1)
    cv2.rectangle(canvas, (max_px, slider_y - SLIDER_H//2),
(SLIDER_RIGHT, slider_y + SLIDER_H//2), tuple(DARK_BLUE), -1)
    cv2.rectangle(canvas, (SLIDER_LEFT, slider_y - SLIDER_H//2),
(SLIDER_RIGHT, slider_y + SLIDER_H//2), (100, 100, 100), 1)
    cv2.circle(canvas, (min_px, slider_y), HANDLE_R,
tuple(DARK_BLUE), -1)
    cv2.circle(canvas, (max_px, slider_y), HANDLE_R,
tuple(DARK_BLUE), -1)
    lbl_min = str(cur_min + 1)
    lbl_max = str(cur_max + 1)
    tw_min, _ = measure_text(lbl_min, 14)
    canvas = put_text(canvas, lbl_min, (min_px - tw_min//2, slider_y
- 22), font_size=14, color=tuple(DARK_BLUE))
    tw_max, _ = measure_text(lbl_max, 14)
    canvas = put_text(canvas, lbl_max, (max_px - tw_max//2, slider_y
- 22), font_size=14, color=tuple(DARK_BLUE))
    canvas = put_text(canvas, "1", (SLIDER_LEFT - 10, slider_y - 6),
font_size=12, color=tuple(DARK_BLUE))
    tw_end, _ = measure_text(str(total), 12)
    canvas = put_text(canvas, str(total), (SLIDER_RIGHT - tw_end +
10, slider_y - 6), font_size=12, color=tuple(DARK_BLUE))
    return canvas

def _sr_idx_to_px(idx):
    if sr_total <= 1:
        return (SLIDER_LEFT + SLIDER_RIGHT) // 2

```



```

        return int(SLIDER_LEFT + (idx / (sr_total - 1)) * (SLIDER_RIGHT
- SLIDER_LEFT))

def _sr_px_to_idx(px):
    if sr_total <= 1:
        return 0
    ratio = (px - SLIDER_LEFT) / (SLIDER_RIGHT - SLIDER_LEFT)
    idx = int(round(ratio * (sr_total - 1)))
    return max(0, min(sr_total - 1, idx))

def _bs_idx_to_px(idx):
    if bs_total <= 1:
        return (SLIDER_LEFT + SLIDER_RIGHT) // 2
    return int(SLIDER_LEFT + (idx / (bs_total - 1)) * (SLIDER_RIGHT
- SLIDER_LEFT))

def _bs_px_to_idx(px):
    if bs_total <= 1:
        return 0
    ratio = (px - SLIDER_LEFT) / (SLIDER_RIGHT - SLIDER_LEFT)
    idx = int(round(ratio * (bs_total - 1)))
    return max(0, min(bs_total - 1, idx))

def _on_slider(mx, my, slider_y):
    return SLIDER_LEFT <= mx <= SLIDER_RIGHT and abs(my - slider_y)
<= HANDLE_R + 15

def _near_handle(px, mx, my, slider_y):
    return abs(mx - px) <= HANDLE_R + 5 and abs(my - slider_y) <=
HANDLE_R + 15

# — Click zones (colunas deslocadas para a direita) —
click_zones = [
    (100, 160, 125, 185, "exe_sr"),
    (100, 210, 125, 235, "exe_bs"),
    (680, 160, 705, 185, "gen_f"),
    (680, 210, 705, 235, "gen_m"),
    (840, 160, 865, 185, "side_esq"),
    (840, 210, 865, 235, "side_dir")
]

btn_calc = (1075, 178, 1195, 218)
hover_calc = False

# — Mouse callback —
def _mouse_callback(event, x, y, flags, param):

```

```

        nonlocal sr_min, sr_max, bs_min, bs_max, dragging, hover_calc,
sr_full, bs_full
        nonlocal sr_total, bs_total, df_current, current_chart

    if event == cv2.EVENT_LBUTTONDOWN:
        sr_min_px = _sr_idx_to_px(sr_min)
        sr_max_px = _sr_idx_to_px(sr_max)
        bs_min_px = _bs_idx_to_px(bs_min)
        bs_max_px = _bs_idx_to_px(bs_max)

        if _near_handle(sr_min_px, x, y, SR_SLIDER_Y):
            dragging = ("sr", "min"); return
        if _near_handle(sr_max_px, x, y, SR_SLIDER_Y):
            dragging = ("sr", "max"); return
        if _near_handle(bs_min_px, x, y, BS_SLIDER_Y):
            dragging = ("bs", "min"); return
        if _near_handle(bs_max_px, x, y, BS_SLIDER_Y):
            dragging = ("bs", "max"); return
        if _on_slider(x, y, SR_SLIDER_Y):
            click_idx = _sr_px_to_idx(x)
            if abs(click_idx - sr_min) <= abs(click_idx - sr_max):
                sr_min = min(click_idx, sr_max)
            else:
                sr_max = max(click_idx, sr_min)
            return
        if _on_slider(x, y, BS_SLIDER_Y):
            click_idx = _bs_px_to_idx(x)
            if abs(click_idx - bs_min) <= abs(click_idx - bs_max):
                bs_min = min(click_idx, bs_max)
            else:
                bs_max = max(click_idx, bs_min)
            return

    elif event == cv2.EVENT_MOUSEMOVE:
        if dragging is not None:
            ex, handle = dragging
            if ex == "sr":
                new_idx = _sr_px_to_idx(x)
                if handle == "min":
                    sr_min = min(new_idx, sr_max)
                else:
                    sr_max = max(new_idx, sr_min)
            else:
                new_idx = _bs_px_to_idx(x)
                if handle == "min":
                    bs_min = min(new_idx, bs_max)

```

```

        else:
            bs_max = max(new_idx, bs_min)
            return
        hover_calc = (btn_calc[0] <= x <= btn_calc[2] and
                      btn_calc[1] <= y <= btn_calc[3])
        return

    elif event == cv2.EVENT_LBUTTONUP:
        if dragging is not None:
            dragging = None
        return

    if event != cv2.EVENT_LBUTTONDOWN:
        return

    # Botão Calcular -- rebuild chart from current slider/filter
    state
    if btn_calc[0] <= x <= btn_calc[2] and btn_calc[1] <= y <=
    btn_calc[3]:
        df_current = _build_filtered_data()
        current_chart = _generate_processed_chart(df_current)
        return

    # Checkboxes
    for x1, y1, x2, y2, key in click_zones:
        if x1 <= x <= x2 and y1 <= y <= y2:
            if key == "exe_sr" and opts["exe_sr"] and not
            opts["exe_bs"]: continue
            if key == "exe_bs" and opts["exe_bs"] and not
            opts["exe_sr"]: continue
            if key == "gen_f" and opts["gen_f"] and not
            opts["gen_m"]: continue
            if key == "gen_m" and opts["gen_m"] and not
            opts["gen_f"]: continue
            if key == "side_esq" and opts["side_esq"] and not
            opts["side_dir"]: continue
            if key == "side_dir" and opts["side_dir"] and not
            opts["side_esq"]: continue
            opts[key] = not opts[key]
            break

    while not wm.should_close:
        img = blank_canvas()
        cv2.rectangle(img, (BORDER_INSET, 80), (W - BORDER_INSET, H -
        BORDER_INSET), DARK_BLUE, 2)

```

```

# Título
title_text = translate("Visualize Data")
font_size_title = 43
tw, th = measure_text(title_text, font_size_title, is_bold=True)
tx = (W - tw) // 2
ty = 80 - th // 2 - 10
pad = 14
cv2.rectangle(img, (tx - pad, ty - pad), (tx + tw + pad, ty + th
+ pad), BG, -1)
img = put_text(img, title_text, (tx, ty),
font_size=font_size_title, color=tuple(DARK_BLUE), is_bold=True)

# Faixa de Instrução Principal
cv2.rectangle(img, (80, 110), (1070, 145), tuple(DARK_BLUE), -1)
img = put_text(img, translate("choose"), (95, 118),
font_size=15, color=(255, 255, 255))

# Checkboxes
img = _draw_chk(img, 100, 160, opts["exe_sr"], translate("Sit
and Reach"))
img = _draw_chk(img, 100, 210, opts["exe_bs"], translate("Back
Scratch exercise name"))
img = _draw_chk(img, 680, 160, opts["gen_f"],
translate("Feminine"))
img = _draw_chk(img, 680, 210, opts["gen_m"], translate("Male"))
img = _draw_chk(img, 840, 160, opts["side_esq"],
translate("Left"))
img = _draw_chk(img, 840, 210, opts["side_dir"],
translate("Right"))

# Botão Calcular
bc_color = tuple(BTN_BLUE) if hover_calc else (255, 255, 255)
txt_color = (255, 255, 255) if hover_calc else tuple(DARK_BLUE)
cv2.rectangle(img, (btn_calc[0], btn_calc[1]), (btn_calc[2],
btn_calc[3]), tuple(DARK_BLUE), 1)
cv2.rectangle(img, (btn_calc[0]+1, btn_calc[1]+1),
(btn_calc[2]-1, btn_calc[3]-1), bc_color, -1)
calc_text = translate("calculate")
tw_calc, th_calc = measure_text(calc_text, 28)
cx = btn_calc[0] + (btn_calc[2] - btn_calc[0] - tw_calc) // 2
cy = btn_calc[1] + (btn_calc[3] - btn_calc[1] - th_calc) // 2
img = put_text(img, calc_text, (cx, cy), font_size=28,
color=txt_color)

img = _draw_slider(img, _sr_idx_to_px, sr_min, sr_max, sr_total,
SR_SLIDER_Y)

```

```

img = _draw_slider(img, _bs_idx_to_px, bs_min, bs_max, bs_total,
BS_SLIDER_Y)

# — Chart area —
gy = 300
if current_chart is not None:
    gh, gw, _ = current_chart.shape
    gx = (W - gw) // 2
    if not df_current.empty:
        erro_col = next((c for c in ['Erro', 'erro', 'Error'] if
c in df_current.columns), None)
        if erro_col:
            avg_err = df_current[erro_col].mean()
            err_txt = f"{translate('avarage_error')}"
            {avg_err:.2f} cm"

            tw_e, _ = measure_text(err_txt, 14)
            img = put_text(img, err_txt, ((W - tw_e) // 2, gy -
20), font_size=14, color=tuple(DARK_BLUE))
            img[gy:gy+gh, gx:gx+gw] = current_chart
            cv2.rectangle(img, (gx, gy), (gx + gw, gy + gh), DARK_BLUE,
1)

            _content_bottom = gy + gh
        else:
            gx = BORDER_INSET
            cv2.rectangle(img, (gx, gy), (W - gx, H - 120), (255, 255,
255), -1)
            cv2.rectangle(img, (gx, gy), (W - gx, H - 120), DARK_BLUE,
1)

            tw_e, _ = measure_text(translate("no_data"), 16)
            img = put_text(img, translate("no_data"), ((W - tw_e) // 2,
gy + 100), font_size=16, color=tuple(DARK_BLUE))
            _content_bottom = H - 120

    # Rodapé -- centrado entre o fim do conteúdo e o fim do
retângulo exterior
    _outer_bottom = H - BORDER_INSET
    _gap = _outer_bottom - _content_bottom
    _esc_text = translate("esc_return")
    _tw_esc, _th_esc = measure_text(_esc_text, 26)
    _esc_y = _content_bottom + _gap // 2 - _th_esc // 2
    img = put_text(img, _esc_text, ((W - _tw_esc) // 2, _esc_y),
font_size=26, color=tuple(DARK_BLUE))

wm.show(img)
key = wm.poll()
if key == "close":

```

```
break
```

```
wm.close()
```

exercises/sit_and_reach.py

Executa o exercício de Sentar e Alcançar 2 vezes para cada lado, 4 no total, utiliza o MediaPipe Holistic e Kinect. Mede a distância relativa da mão com o pé, por meio de visão computacional, primeiro calibra para registrar a posição do pé, depois mede o alcance da mão.

Faz as importações necessárias;

```
import time

import cv2
import mediapipe as mp
import numpy as np
from locale_setup import translate
from utils import (
    ReturnToMenu,
    WindowManager,
    calculate_angle,
    calculate_distance_2d,
    draw_angle_arc,
    rolling_average,
    read_kinect_frame,
    append_to_excel,
    append_to_log,
)
from ui.exercise_intro import show_exercise_intro
from ui.forms import (
    show_real_distance_screen_styled,
    show_repetition_result,
    show_exercise_final,
    show_register_screen_styled,
    _make_base,
)
from ui.draw import put_text, put_text_multi, measure_text
from ui.theme import W, H, HEADER_H

from config import (
    SIT_AND_REACH_PIXEL_TO_CM,
    SAR_CALIB_ELBOW_MIN, SAR_CALIB_ELBOW_MAX,
    SAR_CALIB_HIP_MIN, SAR_CALIB_HIP_MAX,
    SAR_CALIB_KNEE_MIN, SAR_CALIB_KNEE_MAX,
    SAR_POSTURE_ELBOW_MIN, SAR_POSTURE_ELBOW_MAX,
```

```
SAR_POSTURE_HIP_MIN, SAR_POSTURE_HIP_MAX,
SAR_POSTURE_KNEE_MIN, SAR_POSTURE_KNEE_MAX,
SAR_OPP_ELBOW_MIN, SAR_OPP_ELBOW_MAX,
SAR_OPP_KNEE_MIN, SAR_OPP_KNEE_MAX,
SAR_CALIBRATION_DURATION, SAR_POSE_DURATION,
SAR_AVERAGE_OVER, SAR_ERROR_RIGHT, SAR_ERROR_LEFT,
SAR_SIGN_THRESHOLD,
```

Faz o setup do MediaPipe e das posições de referência;

```
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
mp_holistic = mp.solutions.holistic

# Landmark index sets: [shoulder, elbow, wrist, hip, knee, ankle,
#                       opp_hip, opp_knee, opp_ankle,
#                       opp_shoulder, opp_elbow, opp_wrist]
# repeats 0-1 → right leg extended (use left-side arm landmarks)
# repeats 2-3 → left leg extended (use right-side arm landmarks)
_POSE_INDICES = {
    "right": [11, 13, 15, 23, 25, 27, 24, 26, 28, 12, 14, 16],
    "left": [12, 14, 16, 24, 26, 28, 23, 25, 27, 11, 13, 15],
}

# Foot landmark used to anchor the reference point
_FOOT_INDEX = {"right": 31, "left": 32}

# Hand offset adjustments (pixels) per side to improve tip accuracy
# Original: {"right": (+5, +8), "left": (-5, +8)}
_HAND_OFFSET = {"right": (0, 0), "left": (0, 0)}
```

Faz as funções para verificar o lado, pegar as posições de referência, processá-las, desenhá-las, calcular os ângulos e compará-los, desenhá-los, checar a posição da calibração, depois checa a postura;

```
def _side(repeats):
    return "right" if repeats in (0, 1) else "left"

def _get_landmarks(results, repeats):
    """Return (pose_landmarks, hand_landmarks) or (None, None)."""
    side = _side(repeats)
    if side == "right":
        hand_result = results.left_hand_landmarks
    else:
        hand_result = results.right_hand_landmarks

    if not results.pose_landmarks or not hand_result:
        return None, None
```

```

    return results.pose_landmarks.landmark, hand_result.landmark

def _process_landmarks(results, repeats):
    """Return landmarks only when all required joints are visible."""
    pose_lm, hand_lm = _get_landmarks(results, repeats)
    if pose_lm is None:
        return None, None

    side = _side(repeats)
    required = [pose_lm[i] for i in _POSE_INDICES[side]]

    if all(lm.visibility > 0.0 for lm in required):
        return pose_lm, hand_lm
    return None, None

def _draw_landmarks(image, results, repeats):
    side = _side(repeats)
    mp_drawing.draw_landmarks(
        image, results.pose_landmarks, mp_holistic.POSE_CONNECTIONS,

    landmark_drawing_spec=mp_drawing_styles.get_default_pose_landmarks_style
    ())
    hand_lm = results.left_hand_landmarks if side == "right" else
    results.right_hand_landmarks
    mp_drawing.draw_landmarks(
        image, hand_lm, mp_holistic.HAND_CONNECTIONS,

    landmark_drawing_spec=mp_drawing_styles.get_default_pose_landmarks_style
    ())

def _calculate_angles(repeats, pose_lm):
    """
    Returns (knee, opp_knee, hip, elbow, opp_elbow) angles.
    Untouched from original logic.
    """
    idx = _POSE_INDICES[_side(repeats)]

    def pt(i):
        return np.array([pose_lm[idx[i]].x, pose_lm[idx[i]].y])

    shoulder, elbow, wrist = pt(0), pt(1), pt(2)
    hip, knee, ankle = pt(3), pt(4), pt(5)
    opp_hip, opp_knee, opp_ankle = pt(6), pt(7), pt(8)

```



```

opp_shoulder, opp_elbow, opp_wrist = pt(9), pt(10), pt(11)

return (
    calculate_angle(hip,      knee,      ankle),
    calculate_angle(opp_hip,   opp_knee,  opp_ankle),
    calculate_angle(shoulder,  hip,       knee),
    calculate_angle(shoulder,  elbow,     wrist),
    calculate_angle(opp_shoulder, opp_elbow, opp_wrist),
)

def _draw_angle_arcs(repeats, knee, opp_knee, hip, elbow, opp_elbow,
                    pose_lm, image, frame):
    """Overlay angle arcs and text on *image* using Unicode wrapper.
    Returns the modified image."""
    side = _side(repeats)
    idx = _POSE_INDICES[side]

    def to_px(i):
        return tuple(np.multiply(
            [pose_lm[idx[i]].x, pose_lm[idx[i]].y],
            [frame.shape[1], frame.shape[0]]
        ).astype(int))

    sh, el, wr = to_px(0), to_px(1), to_px(2)
    hp, kn, an = to_px(3), to_px(4), to_px(5)
    o_sh, o_el, o_wr = to_px(9), to_px(10), to_px(11)

    draw_angle_arc(image, hp, kn, an, knee)
    draw_angle_arc(image, sh, hp, kn, hip)
    draw_angle_arc(image, sh, el, wr, elbow)
    draw_angle_arc(image, o_sh, o_el, o_wr, opp_elbow)

    return put_text_multi(image, [
        (f"{translate('Knee Angle')}: {knee:.1f}", kn, 24,
        (0, 230, 0), False),
        (f"{translate('Hip Angle')}: {hip:.1f}", hp, 24,
        (0, 235, 0), False),
        (f"{translate('Elbow Angle')}: {elbow:.1f}", el, 24,
        (0, 235, 0), False),
        (f"{translate('Opp Elbow')}: {opp_elbow:.1f}", o_el, 24,
        (0, 235, 0), False),
        (f"{translate('Opp Knee')}: {opp_knee:.1f}", (1000, 400), 24,
        (0, 235, 0), False),
    ])

```

```

def _check_calibration(calib_time, foot, repeats, iw, ih,
                        knee, opp_knee, hip, elbow,
                        progress, locked, duration, pose_lm):
    """
    Returns (status, progress, calib_time, foot, locked).
    Untouched calibration logic from original.
    """
    side = _side(repeats)
    foot_index = _FOOT_INDEX[side]

    in_position = (
        SAR_CALIB_KNEE_MIN < knee < SAR_CALIB_KNEE_MAX and
        SAR_CALIB_HIP_MIN < hip < SAR_CALIB_HIP_MAX and
        SAR_CALIB_ELBOW_MIN < elbow < SAR_CALIB_ELBOW_MAX and
        locked == 0.0
    )

    if in_position:
        if calib_time is None:
            calib_time = time.time()
        progress = (time.time() - calib_time) / duration
        if progress >= 1.0:
            lm = pose_lm[foot_index]
            foot = (int(lm.x * iw), int(lm.y * ih))
            return "Ok", 1.0, calib_time, foot, 1.0
        return translate("Right Position"), progress, calib_time, None,
0.0

    if locked == 0.0:
        return translate("Wrong Position"), 0.0, None, None, 0.0
    return "Ok", 1.0, calib_time, foot, 1.0

def _check_posture(start_time, knee, opp_knee, hip, elbow, opp_elbow,
                    duration, progress, distance):
    """
    Returns (status, progress, start_time, final_distance_or_None).
    Untouched posture logic from original.
    """
    in_position = (
        SAR_POSTURE_ELBOW_MIN < elbow < SAR_POSTURE_ELBOW_MAX and
        SAR_OPP_ELBOW_MIN < opp_elbow < SAR_OPP_ELBOW_MAX and
        SAR_POSTURE_HIP_MIN < hip < SAR_POSTURE_HIP_MAX and
        SAR_POSTURE_KNEE_MIN < knee < SAR_POSTURE_KNEE_MAX
    )

```

```

    if in_position:
        if start_time is None:
            start_time = time.time()
        progress = (time.time() - start_time) / duration
        if progress >= 1.0:
            return translate("Correct"), 1.0, start_time, -distance
        return translate("Correct"), min(progress, 1.0), start_time,
None

    return translate("Incorrect"), 0.0, None, None

```

Função do loop principal, em cada frame: aguarda um frame colorido, lê o frame, processa as posições de referência, se são válidas: desenha o esqueleto, calcula os ângulos, desenha os arcos, desenha os círculos, verifica a calibração (se sair da posição reseta), quando conclui a calibração, calcula a distância entre mão e pé, converte px em cm, verifica a postura, monta o canvas, coloca os textos, etc, e se ESC é premido finaliza;

```

def run_repetition(repeats, kinect, holistic, finish_cb):
    """
    Run one sit-and-reach repetition and return the measured distance
    (str).
    *finish_cb* is called on ESC keypress or fatal error.
    """
    print(f"[run_rep] início repeats={repeats}")

    calib_time    = None
    calib_locked  = 0.0
    calib_prog    = 0.0
    calibration    = translate("Wrong Position")
    foot          = None
    pose_start    = None
    distances     = []
    final_dist    = None
    display_dist  = 0.0

    wm = WindowManager("Sit and Reach", finish_cb, delay=1, size=(W, H))

    while not wm.should_close:
        distance_str = ""
        if not kinect.has_new_color_frame():
            continue

        image, results, frame = read_kinect_frame(kinect, holistic)
        pose_correct = translate("Incorrect")

        pose_lm, hand_lm = _process_landmarks(results, repeats)

```

```

    if pose_lm is not None and hand_lm is not None:
        _draw_landmarks(image, results, repeats)
        angles = _calculate_angles(repeats, pose_lm)
        image = _draw_angle_arcs(repeats, *angles, pose_lm, image,
frame)

        # Visual guides: foot and hand measurement points (image
coords)

        side = _side(repeats)
        ih, iw = image.shape[:2]
        foot_idx = _FOOT_INDEX[side]
        foot_pt = (int(pose_lm[foot_idx].x * iw),
int(pose_lm[foot_idx].y * ih))
        hand_pt = (int(hand_lm[12].x * iw), int(hand_lm[12].y *
ih))

        cv2.circle(image, foot_pt, 12, (0, 255, 255), -1)
        cv2.circle(image, hand_pt, 12, (0, 255, 255), -1)

        (calibration, calib_prog, calib_time,
        foot, calib_locked) = _check_calibration(
            calib_time, foot, repeats, iw, ih,
            *angles[:4], calib_prog, calib_locked,
            SAR_CALIBRATION_DURATION, pose_lm,
        )

        if calibration == "Ok":
            ox, oy = _HAND_OFFSET[_side(repeats)]
            hand = (int(hand_lm[12].x * iw) + ox,
                    int(hand_lm[12].y * ih) + oy)

            dist_px = calculate_distance_2d(hand, foot)
            distance = dist_px * SIT_AND_REACH_PIXEL_TO_CM

            distances.append(distance)
            if len(distances) > SAR_AVERAGE_OVER:
                distances.pop(0)
                distance = rolling_average(distances)

            pose_correct, _prog, pose_start, final_dist =
_check_posture(
                pose_start, *angles, SAR_POSE_DURATION, 0, distance
            )

            display_dist = distance
            if _side(repeats) == "right" and hand[0] < foot[0] and

```

```

distance > SAR_SIGN_THRESHOLD:
    display_dist = -(distance + SAR_ERROR_RIGHT)
    elif _side(repeats) == "left" and hand[0] > foot[0] and
distance > SAR_SIGN_THRESHOLD:
    display_dist = -(distance + SAR_ERROR_LEFT)

    if final_dist is not None:
        if side == "right" and hand[0] < foot[0] and
distance > SAR_SIGN_THRESHOLD:
            final_dist = -(final_dist + SAR_ERROR_RIGHT)
            elif side == "left" and hand[0] > foot[0] and
distance > SAR_SIGN_THRESHOLD:
            final_dist = -(final_dist + SAR_ERROR_LEFT)
        break

    image = put_text(image, f"{translate('Foot')}:
{foot[0]}, {foot[1]}", (1000, 100), font_size=24, color=(0, 235, 0))
    image = put_text(image, f"{translate('Hand')}:
{hand[0]}, {hand[1]}", (1000, 200), font_size=24, color=(0, 235, 0))

    side_label = translate("right_side_label") if repeats in (0, 1)
else translate("left_side_label")
    rep_num = (repeats % 2) + 1

    canvas = _make_base(translate("Sit and Reach"), side_label,
rep_num, 2)

    feed_h = H - HEADER_H - 10
    feed_w = int(image.shape[1] * feed_h / image.shape[0])
    feed_x = (W - feed_w) // 2
    resized = cv2.resize(image, (feed_w, feed_h))
    canvas[HEADER_H + 5 : HEADER_H + 5 + feed_h, feed_x : feed_x +
feed_w] = resized

    overlay = canvas.copy()
    sar_box_x = (W - 415) // 2
    cv2.rectangle(overlay, (sar_box_x, HEADER_H + 10), (sar_box_x +
415, HEADER_H + 80), (0, 0, 0), -1)
    cv2.addWeighted(overlay, 0.5, canvas, 0.5, 0, canvas)

    if calibration == "Ok":
        _l1 = f"{translate('Pose')}: {pose_correct}"
        sign_char = "-" if display_dist < 0 else "+"
        _l2 = f"{translate('Dist')}:
{sign_char}{abs(display_dist):.2f} cm"

```

```

        _tw1, _th = measure_text(_l1, 24)
        canvas = put_text(canvas, _l1, (sar_box_x + (415 - _tw1) //
2, HEADER_H + 15), font_size=24, color=(255, 255, 255))
        _tw2, _th = measure_text(_l2, 24)
        canvas = put_text(canvas, _l2, (sar_box_x + (415 - _tw2) //
2, HEADER_H + 43), font_size=24, color=(255, 255, 255))
    else:
        _cal = f"{translate('Calibration')}: {calibration}"
        _tw, _th = measure_text(_cal, 24)
        canvas = put_text(canvas, _cal, (sar_box_x + (415 - _tw) //
2, HEADER_H + 10 + (70 - _th) // 2), font_size=24, color=(255, 255,
255))

    wm.show(canvas)

    key = wm.poll()
    if key == "close":
        finish_cb()
        return None

    return round(final_dist, 2)

```

Salva os dados nas tabelas e no log;

```

_EXCEL_PATH = "./arquivos/tabelas_testes/sit_and_reach_test_julia.xlsx"
_LOG_PATH   = "./arquivos/logs_utentes/logs_sit_and_reach_utentes"

```

Função contém o entry point público. Começa com a tela de registo e salva os dados, loop 4x, **show_exercise_intro()**: instrução com o número da repetição, **dist = run_repetition()**: loop principal, retorna distância medida, determina o lado baseado na repetição, **dist = run_repetition()**: tela para inserir a medida real medida manualmente, cálculo de erro: **dist = run_repetition**, salva no excel e no log, salva no array para a tela final e por fim mostra a tela final com todas as medidas;

```

def run(kinect, holistic, finish_cb):
    """
    Run 4 sit-and-reach repetitions (2 per leg).

    Parameters
    -----
    kinect      : PyKinectRuntime instance
    holistic    : MediaPipe Holistic instance
    participant : dict with keys age, height, weight, gender
    finish_cb   : callable -- called to terminate the program cleanly
    """
    print("[SAR run] início")
    try:
        print("[SAR run] a chamar show_register_screen_styled...")

```

```

        age, height, weight, gender_raw = show_register_screen_styled(
            translate("Sit and Reach"), translate("right_side_label"),
1, 2, finish_cb
        )
        participant = {
            "age": age,
            "height": height,
            "weight": weight,
            "gender": "Feminine" if gender_raw.strip().upper() == "F"
else "Male",
        }

        print(f"[SAR run] cadastro completo: age={age},
height={height}")

        distances_right, distances_left = [], []
        reals_right, reals_left = [], []

        for rep in range(4):
            print(f"[SAR run] rep={rep} a iniciar")

            show_exercise_intro("Sit and Reach", rep, finish_cb)
            print(f"[SAR run] rep={rep} intro concluído -- a chamar
run_repetition")

            dist = run_repetition(rep, kinect, holistic, finish_cb)
            print(f"[SAR run] rep={rep} dist={dist}")

            if dist is None:
                print(translate("Exercise not performed correctly. "))
                finish_cb()
                return

            side = "right" if rep in (0, 1) else "left"
            side_label = translate("right_side_label") if rep in (0, 1)
else translate("left_side_label")
            real = show_real_distance_screen_styled(
                translate("Sit and Reach"), side_label, (rep % 2) + 1,
2, finish_cb
            )
            if side == "right":
                reals_right.append(real)
            else:
                reals_left.append(real)
            error = abs(abs(float(real)) - abs(dist))

```

```

        append_to_excel(_EXCEL_PATH, {
            "Age": participant["age"], "Height":
participant["height"],
            "Weight": participant["weight"], "Gender":
participant["gender"],
            "Side": side,
            "Real distance": real, "Calculated distance": dist,
"Erro": error,
        })
        append_to_log(_LOG_PATH,
            participant["age"], participant["height"],
            participant["weight"], participant["gender"],
            real, dist, side)

        if side == "right":
            distances_right.append(dist)
        else:
            distances_left.append(dist)

        show_repetition_result(
            translate("Sit and Reach"), side_label, (rep % 2) + 1,
2,
            f"{dist:.2f}", real, error, finish_cb
        )

        best_right = max(distances_right)
        best_left = max(distances_left)
        errors_right = [abs(abs(float(reals_right[i])) -
abs(float(distances_right[i]))) for i in range(2)]
        errors_left = [abs(abs(float(reals_left[i])) -
abs(float(distances_left[i]))) for i in range(2)]
        show_exercise_final(
            translate("Sit and Reach"),
            f"{distances_right[0]:.2f}", f"{distances_right[1]:.2f}",
            f"{distances_left[0]:.2f}", f"{distances_left[1]:.2f}",
            reals_right[0], reals_right[1],
            reals_left[0], reals_left[1],
            errors_right[0], errors_right[1],
            errors_left[0], errors_left[1],
            finish_cb
        )
    except ReturnToMenu:
        cv2.destroyAllWindows()
        return
    except Exception as e:
        import traceback, sys

```



```
traceback.print_exc(file=sys.stdout)
sys.stdout.flush()
raise
```

exercises/back_scratch.py

Executa o exercício de Alcançar atrás das Costas 2 vezes para cada lado, 4 no total, utiliza o MediaPipe Holistic e Kinect. Mede a distância relativa dos dedos do meio, por meio de visão computacional.

Faz as importações necessárias;

```
import time
import cv2
import mediapipe as mp

from locale_setup import translate
from utils import (
    ReturnToMenu,
    WindowManager,
    calculate_distance_2d,
    read_kinect_frame,
    append_to_excel,
    append_to_log,
)
from ui.exercise_intro import show_exercise_intro
from ui.forms import (
    show_real_distance_screen_styled,
    show_repetition_result,
    show_exercise_final,
    show_register_screen_styled,
    _make_base,
)
from ui.draw import put_text, measure_text
from ui.theme import W, H, HEADER_H

from config import (
    BACK_SCRATCH_PIXEL_TO_CM,
    BS_DISTANCE_THRESHOLD,
    BS_POSE_HELD_DURATION,
    BS_POSE_NO_HELD_DURATION,
    BS_ERROR,
)
```

Faz setup do MediaPipe;

```
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
```

```
mp_holistic = mp.solutions.holistic
```

Faz as funções para usar no loop, verificar qual o lado, desenhar as posições de referência, desenhar os dedos do meio, criar o timer quando a posição das mãos estiver correta;

```
def _side(repeats):  
    return "right" if repeats in (0, 1) else "left"
```

```
def _draw_landmarks(image, results):  
    for hand_lm in (results.left_hand_landmarks,  
results.right_hand_landmarks):  
        mp_drawing.draw_landmarks(  
            image, hand_lm, mp_holistic.HAND_CONNECTIONS,
```

```
landmark_drawing_spec=mp_drawing_styles.get_default_hand_landmarks_style  
())
```

```
def _draw_middle_finger_both(image, results):  
    middle_indices = [9, 10, 11, 12]  
    h, w, _ = image.shape  
    color = (0, 255, 255)  
  
    def draw_one(hand_lm):  
        pts = []  
        for i in middle_indices:  
            x = int(hand_lm.landmark[i].x * w)  
            y = int(hand_lm.landmark[i].y * h)  
            pts.append((x, y))  
            cv2.circle(image, (x, y), 6, color, -1)  
            cv2.circle(image, (x, y), 6, (255, 255, 255), 1)  
        for a, b in zip(pts, pts[1:]):  
            cv2.line(image, a, b, color, 2)  
        return pts[-1] # tip (landmark 12)
```

```
    tip_left = draw_one(results.left_hand_landmarks) if  
results.left_hand_landmarks else None  
    tip_right = draw_one(results.right_hand_landmarks) if  
results.right_hand_landmarks else None
```

```
    if tip_left and tip_right:  
        cv2.line(image, tip_left, tip_right, (0, 0, 255), 3)
```

```
def _check_distance_timer(distance, start_time):  
    """
```

```

Accumulate time while hands are within threshold.
Returns (elapsed_seconds, updated_start_time).
"""
if distance < BS_DISTANCE_THRESHOLD:
    if start_time is None:
        start_time = time.time()
    return time.time() - start_time, start_time
return 0, None

```

Função para executar loop principal, em cada frame: aguarda um frame, lê o frame, se detecta ambas as mãos, atualiza o timer, desenha as posições de referência, calcula em px, converte em cm, acumula tempo, se uma mão não está visível reseta o timer e espera, monta o canvas com as bases, textos e formas;

```

def run_repetition(repeats, kinect, holistic, finish_cb):
    print(f"[run_rep] início repeats={repeats}")

    start_time      = None
    last_detected   = time.time()
    final_dist      = None
    elapsed         = 0.0

    side_label      = translate("right_side_label") if repeats in (0, 1)
else translate("left_side_label")
    rep_num         = (repeats % 2) + 1

    wm = WindowManager("Back Scratch", finish_cb, delay=1, size=(W, H))

    while not wm.should_close:
        if not kinect.has_new_color_frame():
            continue

        image, results, _ = read_kinect_frame(kinect, holistic)
        pose_correct = translate("Incorrect")

        if results.left_hand_landmarks and results.right_hand_landmarks:
            last_detected = time.time()
            _draw_landmarks(image, results)
            _draw_middle_finger_both(image, results)
            pose_correct = translate("Correct")

            ih, iw = image.shape[:2]
            lm_left  = results.left_hand_landmarks.landmark[12]
            lm_right = results.right_hand_landmarks.landmark[12]

            left_hand  = (int(lm_left.x * iw), int(lm_left.y * ih))
            right_hand = (int(lm_right.x * iw), int(lm_right.y * ih))

```

```

        dist_px = calculate_distance_2d(left_hand, right_hand)
        distance = (dist_px * BACK_SCRATCH_PIXEL_TO_CM) - BS_ERROR

        elapsed, start_time = _check_distance_timer(distance,
start_time)

        if elapsed >= BS_POSE_HELD_DURATION:
            final_dist = round(-distance, 2)
            break
    else:
        if time.time() - last_detected >= BS_POSE_NO_HELD_DURATION:
            start_time = None

    # --- Infraestrutura Visual Espelhada do Sit and Reach ---
    canvas = _make_base(translate("Back Scratch exercise name"),
side_label, rep_num, 2)

    # Dimensionamento Proporcional da Imagem da Câmera
    feed_h = H - HEADER_H - 10
    feed_w = int(image.shape[1] * feed_h / image.shape[0])
    feed_x = (W - feed_w) // 2
    resized = cv2.resize(image, (feed_w, feed_h))
    canvas[HEADER_H + 5 : HEADER_H + 5 + feed_h, feed_x : feed_x +
feed_w] = resized

    # Overlay Translúcido Superior para Caixa de Status (centrado)
    overlay = canvas.copy()
    bs_box_x = (W - 415) // 2
    cv2.rectangle(overlay, (bs_box_x, HEADER_H + 10), (bs_box_x +
415, HEADER_H + 80), (0, 0, 0), -1)
    cv2.addWeighted(overlay, 0.5, canvas, 0.5, 0, canvas)

    if pose_correct == translate("Correct"):
        _l1 = f"{translate('Pose')}: {pose_correct}"
        _l2 = f"{translate('Hold')}: {elapsed:.1f}s /
{BS_POSE_HELD_DURATION}s" if elapsed > 0 else f"{translate('Hold')}:
---"

        _tw1, _th = measure_text(_l1, 24)
        canvas = put_text(canvas, _l1, (bs_box_x + (415 - _tw1) //
2, HEADER_H + 15), font_size=24, color=(255, 255, 255))
        _tw2, _th = measure_text(_l2, 24)
        canvas = put_text(canvas, _l2, (bs_box_x + (415 - _tw2) //
2, HEADER_H + 43), font_size=24, color=(255, 255, 255))
    else:
        _cal = f"{translate('Calibration')}: ---"

```

```

        _tw, _th = measure_text(_cal, 24)
        canvas = put_text(canvas, _cal, (bs_box_x + (415 - _tw) //
2, HEADER_H + 10 + (70 - _th) // 2), font_size=24, color=(255, 255,
255))

    wm.show(canvas)

    key = wm.poll()
    if key == "close":
        finish_cb()
        return None

    return final_dist

```

Pega os diretórios das tabelas e log para salvar os dados;

```

_EXCEL_PATH = "./arquivos/tabelas_testes/back_scratch_test_julia.xlsx"
_LOG_PATH   = "./arquivos/logs_utentes/logs_back_scratch_utentes"

```

Função para executar o loop principal. Executa a tela de registo, no loop mostra a intro, executa **run_repetition()**, mostra a tela para medida rela, salva nas tabelas e no log, mostra a tela de resumo da repetição e por fim mostra a tela com todos os exercícios;

```

def run(kinect, holistic, finish_cb):
    print("[BS run] início")
    try:
        print("[BS run] a chamar show_register_screen_styled...")
        age, height, weight, gender_raw = show_register_screen_styled(
            translate("Back Scratch exercise name"),
            translate("right_side_label"), 1, 2, finish_cb
        )
        participant = {
            "age": age,
            "height": height,
            "weight": weight,
            "gender": "Feminine" if gender_raw.strip().upper() == "F"
        else "Male",
        }

        print(f"[BS run] cadastro completo: age={age}, height={height}")

        distances_right, distances_left = [], []
        reals_right, reals_left = [], []

        for rep in range(4):
            print(f"[BS run] rep={rep} a iniciar")

            show_exercise_intro("Back Scratch", rep, finish_cb)
            print(f"[BS run] rep={rep} intro concluído -- a chamar

```

```

run_repetition")

    dist = run_repetition(rep, kinect, holistic, finish_cb)
    print(f"[BS run] rep={rep} dist={dist}")

    if dist is None:
        print(translate("Exercise not performed correctly. "))
        finish_cb()
        return

    side = "right" if rep in (0, 1) else "left"
    side_label = translate("right_side_label") if rep in (0, 1)
else translate("left_side_label")

    real = show_real_distance_screen_styled(
        translate("Back Scratch exercise name"), side_label,
(rep % 2) + 1, 2, finish_cb
    )
    if side == "right":
        reals_right.append(real)
    else:
        reals_left.append(real)
    error = abs(abs(float(real)) - abs(dist))

    append_to_excel(_EXCEL_PATH, {
        "Age": participant["age"], "Height":
participant["height"],
        "Weight": participant["weight"], "Gender":
participant["gender"],
        "Side": side,
        "Real distance": real, "Calculated distance": dist,
"Erro": error,
    })
    append_to_log(_LOG_PATH,
        participant["age"], participant["height"],
        participant["weight"], participant["gender"],
        real, dist, side)

    if side == "right":
        distances_right.append(dist)
    else:
        distances_left.append(dist)

    show_repetition_result(
        translate("Back Scratch exercise name"), side_label,
(rep % 2) + 1, 2,

```

```

        f"{dist:.2f}", real, error, finish_cb
    )

    while len(distances_right) < 2: distances_right.append(0.0)
    while len(distances_left) < 2: distances_left.append(0.0)
    while len(reals_right) < 2:     reals_right.append(0.0)
    while len(reals_left) < 2:     reals_left.append(0.0)

    errors_right = [abs(abs(float(reals_right[i])) -
abs(float(distances_right[i]))) for i in range(2)]
    errors_left  = [abs(abs(float(reals_left[i])) -
abs(float(distances_left[i]))) for i in range(2)]
    show_exercise_final(
        translate("Back Scratch exercise name"),
        f"{distances_right[0]:.2f}", f"{distances_right[1]:.2f}",
        f"{distances_left[0]:.2f}", f"{distances_left[1]:.2f}",
        reals_right[0], reals_right[1],
        reals_left[0],  reals_left[1],
        errors_right[0], errors_right[1],
        errors_left[0],  errors_left[1],
        finish_cb
    )
    cv2.destroyAllWindows()

except ReturnToMenu:
    cv2.destroyAllWindows()
    return
except Exception as e:
    import traceback, sys
    traceback.print_exc(file=sys.stdout)
    sys.stdout.flush()
    raise

```

locale/

```

locale/
├── pt_PT/
│   ├── LC_MESSAGES/
│   │   ├── messages.po ← ficheiro fonte editável (português)
│   │   └── messages.mo ← compilado binário (lido pelo gettext)
└── en_US/
    ├── LC_MESSAGES/
    │   ├── messages.po
    │   └── messages.mo

```

`locale_setup.py` carrega `gettext()`:

- `set_language()`: abre o ficheiro `.mo` para configurar a língua;

- `translate()`: chama `gettext()` com o `msgid`, e retorna a tradução ou o próprio `id` em caso de erro;

Exemplo no arquivo: `msgid` - chave em inglês usada no código, `msgstr` - tradução para o idioma;

```
msgid "Sit and Reach"
msgstr "Sentar e Alcançar"      ← pt_PT
msgstr "Sit and Reach"         ← en_US
```

O Python `gettext` lê `.mo` (binário), não `.po` (texto), por isso é necessário compilar, como descrito na instalação.

camera.py

A `camera.py` substitui o `PyKinectRuntime` do Kinect V2 por um adaptador para a Orbbec Femto Mega usando o `pyorbbecsdk2`.

Faz as importações necessárias;

```
import cv2
import numpy as np
from pyorbbecsdk import Pipeline, Context, OLogLevel
```

Função Impede que o Orbbec SDK crie o ficheiro `Log/OrbbecSDK.log.txt` e polua a consola. Cria um `Context` (objeto global do SDK) e define ambos os níveis para `NONE` (desligado).

```
def _disable_sdk_logging():
    ctx = Context()
    ctx.set_logger_to_console(OLogLevel.NONE)
    ctx.set_logger_to_file(OLogLevel.NONE, "")
```

Função converte os dados MJPG comprimidos (vindos da câmara) para um array BGRA (4 canais): `cv2.imdecode(data, cv2.IMREAD_COLOR)` = descodifica o JPEG para BGR (3 canais)

`cv2.cvtColor(bgr, cv2.COLOR_BGR2BGRA)` = adiciona o canal alpha, resultando em BGRA

Isto é necessário porque o resto do projeto (`utils.py`, `exercises/`) espera BGRA (o formato original do Kinect V2).

```
def _mjpg_to_bgra(data):
    bgr = cv2.imdecode(data, cv2.IMREAD_COLOR)
    return cv2.cvtColor(bgr, cv2.COLOR_BGR2BGRA)
```

Classe desliga o logging do SDK, cria um `Pipeline` e inicia com a configuração padrão (lê `OrbbecSDKConfig.xml`), `self._last_frame`, buffer onde guarda o último frame recebido, inicializado como imagem preta 1920×1080 BGRA

O `Pipeline.start()` sem `Config` usa a configuração XML incluída no SDK, que já sabe os formatos suportados pela Femto Mega.

```
def __init__(self):
    _disable_sdk_logging()
    self.pipeline = Pipeline()
    self.pipeline.start()
```



```
self._last_frame = np.zeros((1080, 1920, 4), dtype=np.uint8)
```

Função que substitui o PyKinectRuntime.has_new_color_frame() original:

wait_for_frames(1000): bloqueia até 1000 ms à espera de um conjunto sincronizado de frames (color + depth). Retorna None se timeout.

get_color_frame(): extrai o frame de cor do conjunto

frame.get_data() → **np.asanyarray()** → **_mjpg_to_bgra()** — descodifica MJPG para BGRA e guarda em **self._last_frame**, retorna True se conseguiu, False se não

```
def has_new_color_frame(self):
    frames = self.pipeline.wait_for_frames(1000)
    if not frames:
        return False
    frame = frames.get_color_frame()
    if not frame:
        return False
    self._last_frame = _mjpg_to_bgra(np.asanyarray(frame.get_data()))
    return True
```

Função devolve o último frame BGRA guardado (array numpy (H, W, 4)). Chamado pelo **read_kinect_frame()** em [utils.py.close\(\)](#)

```
def get_last_color_frame(self):
    return self._last_frame
```

Função para o pipeline e liberta a câmara. Chamado em runner.py ao sair do programa.

```
def close(self):
    self.pipeline.stop()
```